

Commodore CDTV

U62 front-panel controller

Sanyo LC6554H · ROM version 1.20, 26 September 1990

Complete annotated disassembly

Instruction set from the Sanyo LC6554D/H datasheet, EN 2156B, pages 24–26

Signal names from the CDTV Service Manual glossary and PinOut-U62.txt

U75 interface from U75_pins.txt

Contents

Reset and initialisation

\$000 [f_Reset](#) — power-on entry. Kill the control register, park the U75 command inputs, wipe RAM, start the real-time clock, set a default volume, then start the display.

\$400 [f_DefaultVolume](#) — choose the startup attenuation - \$12 (a moderate level) if the main

Interrupt vectors and main loop

\$038 [v_TimerInterrupt](#) —

\$03C [v_SecondInterrupt](#) — this firmware never enables that source

\$03D [f_MainLoop](#) — keep the clock readout fresh continuously, and run everything else once every sixteen display ticks.

Display

\$098 [f_RefreshDigit](#) — light exactly one of the eight VFD digits, then set up for the next.

Front-panel keys

\$104 [f_ScanKeys](#) — strobe all four key columns, merge in the virtual keys U75 asked for, and work out which keys went down this pass.

\$15E [f_KeyRowPtr](#) — dP -> this row's KEY block, column 0.

\$78E [f_ArmRepeat](#) — start a hold burst: RPT_CMD = which key (1 or 2), RPT_TICK = 0,

\$799 [f_RepeatTick](#) — deliver the rest of a hold burst. Called every service pass; while a

\$7BF [f_RepeatTimeout](#) — one fewer issue left in the burst; at zero, disarm it

Real-time clock and the time display

\$16A [f_ReadClock](#) — read the time of day out of the real-time clock into the display buffer, and set the AM/PM and colon symbols.

\$200 [f_ClockReadHigh](#) — raise the RTC read strobe (PP0 -> U80 pin 2)

\$204 [f_ClockReadLow](#) — drop the RTC read strobe

\$2CE [f_ClockWrite](#) — pulse the RTC write strobe (PI2 -> U74 pin 6)

\$3B2 [f_InitRTC](#) — bring up the MSM6242B real-time clock at power-on.

CD drive

\$41C [f_SendCDCommand](#) — clock the pending CD command out on SDATA/SCK.

\$451 [f_SendCDCommand2](#) — second phase of the CD exchange, eight more SCK pulses

\$479 [f_Wait](#) — a nested busy-wait of about 4.2 ms

\$600 [f_TransportButtons](#) — turn front-panel presses into CD commands and AUS event codes for U75.

\$764 [f_TrackToDigits](#) — turn the drive's reply into the two track digits on the panel.

\$77F [f_PowerSymbol](#) — set TMP1 to 12 (a bar glyph) when the main logic is down,

\$7CB [f_ReadTrackReply](#) — read the two nibbles the CD drive returned and turn them into the track

Audio and volume

\$278 [f_DrawVolumeBar](#) — show the current volume as an eight-step bar across the panel.

\$2BB [f_ShiftBarBit](#) — take the next bar bit out of AC into this digit's Sh

\$300 [f_VolumeKeys](#) — step the volume up or down while a volume key is held.

\$546 [f_SetAttenuator](#) — send the volume to the headphone attenuator on the port G three-wire bus.

\$5B7 [f_ShiftAttBit](#) — clock one bit onto the attenuator bus.

Power, genlock and keyboard reset

\$05F [f_KeyboardReset](#) — pulse the Amiga's _KBRESET line from a front-panel key.

\$208 [f_PowerControl](#) — run the CDTV's soft power switch.

\$2D8 [f_ReadGenlockMode](#) — sample MS0/MS1, the genlock mode select lines the Amiga drives.

\$374 [f_GenlockMode](#) — cycle the genlock mode when asked, and drive GMS0/GMS1.

\$7DB [f_ClearGenlockBits](#) — mask the GMS bits out of the port K shadow, keeping

U75 interface

\$35F [f_LatchU75Requests](#) — turn the level-held U75 request bits into one-shot edges.

\$48A [f_ReadU75Command](#) — read what the Amiga side is asking for.

\$5D5 [f_IssueButton](#) — report a front-panel button press to both sides of the machine.

How to read this listing

```
$0AD 63      RTBL      ; fetch the segment pattern for this digit
$5BA 3F BF   BNC  @skip ; carry clear, so the data bit is 0
$09D 8D      LDZ   #13  ; select port N
```

Colour	Means
\$0AD	ROM address
63	the raw bytes at that address
RTBL	mnemonic
#13	immediate operand
f_Name / @skip	a routine, or a label local to the current routine
; text	what the line is for

Operands are shown symbolically throughout; the numeric address is in the bytes column on the same line. Local labels are named for what they do: **@loop** branches backwards, **@skip** jumps over a few instructions, **@done** reaches the return. They are scoped to the routine they appear in.

On certainty — every address, byte and opcode here is read directly from the ROM image and can be checked against it. The comments are interpretation. Where an interpretation is not settled it says so in the text.

Ports

The LC6554H selects a port with DPL. There is no port H, so slot 7 is unused.

DPL	Port	CDTV signals
0	A	command nibble in from U75 (CPCP0-3 -> U75 PB4-7)
1	B	key matrix returns KI0-2
2	C	key matrix strobes KST0-3
3	D	CD0-3, real-time clock data bus (MSM6242B U61, shared with the Amiga)
4	E	CA0-3, real-time clock register address
5	F	SDATA/SCK serial link
6	G	headphone attenuator VDATA/VCK/VST + _KBRESET
8	I	MS0/MS1 genlock mode from Amiga, PI2 = RTC write strobe via U74, AUPLY
9	J	AUS0-2 event code to U75, MPS power sense
10	K	POWER, GMS0/GMS1 genlock mode to remote, U60 pin 19
11	L	segments Sa-Sd
12	M	segments Se-Sh
13	N	grids G1-G4
14	O	grids G5-G8
15	P	PP0 = RTC read strobe via U80 pin 2

RAM

256 nibbles, addressed as DPH:DPL.

Address	Symbol
\$00	HRS10
\$01	HRS1
\$02	MIN10
\$03	MIN1
\$04	TRK10
\$05	TRK1
\$06	AMPM
\$07	COLON
\$08	DIGIT
\$09	TMP0
\$0A	TMP1
\$0B	TICK
\$0C	COLON_T_LO
\$0D	COLON_T_HI
\$0E	SEC_PREV
\$0F	COLON_ON
\$26	ATTSHADOW
\$29	PORTJ
\$2A	PORTK
\$30	BAR0
\$31	BAR1
\$32	BAR2
\$33	BAR3
\$34	BAR4
\$35	BAR5
\$36	BAR6
\$37	BAR7
\$38	ATTEN_LO
\$39	ATTEN_HI
\$3A	MPS_PREV
\$3B	SEGTMP
\$3C	ATT0
\$3D	ATT1
\$3E	ATT2
\$3F	ATT3
\$40	SCANROW
\$41	CDCMD_LO
\$42	CDCMD_HI
\$43	CDCMD_X
\$45	HOLD2_FIRED

\$47	HOLD3_FIRED
\$48	SEC_NOW
\$50	VKEY0
\$51	VKEY1
\$52	VKEY2
\$53	VKEY3
\$54	SYSREQ
\$55	SYSREQ_PREV
\$56	SYSREQ_NEW
\$57	GMSMODE
\$59	POLL_TICK
\$5A	RPT_CMD
\$5B	RPT_TICK
\$5C	RPT_DELAY
\$5F	GENLOCK_MS
\$60	KBRST_HOLD
\$61	MPS_TMP
\$62	MPS_TMP2
\$10-\$1F	KEY<row>_<n> — four debounce nibbles per key-matrix row: _0 raw, _1 debounced, _2 auto-repeat pulse, _3 hold counter

Reset and initialisation

f_Reset \$000

Purpose power-on entry. Kill the control register, park the U75 command inputs, wipe RAM, start the real-time clock, set a default volume, then start the display.

Outputs all 256 RAM nibbles zeroed; timer loaded with \$7D; interrupts enabled.

Timing runs once. Nothing lights up until SCTL \$1 at \$02D.

```
$000 2C 9F RCTL $F ; silence every control-register bit - interrupts off while RAM is garbage
$002 80 LDZ #0 ; select port A
$003 CF LI #15 ; value 15 = %1111
$004 61 OP ; park the CPCP inputs high so U75 does not see a spurious command
$005 80 LDZ #0 ; point at HRS10
$006 40 LHI #0 ; row 0
@loop:
$007 C0 CLA ; AC = 0
$008 FE XI ; write 0 to this nibble and step to the next column
$009 3E 07 BNZ @loop ; keep going until DPL wraps - one whole RAM row cleared
$00B 23 XAH ; swap AC and the RAM row
$00C 0E INC ; bump the row number
$00D 23 XAH ; swap AC and the RAM row
$00E 3E 07 BNZ @loop ; repeat until all 16 rows are done
$010 AB B2 CAL f_InitRTC ; initialise the real-time clock
$012 C2 LI #2 ; AC = 2
$013 89 LDZ #9 ; point at PORTJ
$014 42 LHI #2 ; row 2
$015 02 S ; PORTJ = 2
$016 89 LDZ #9 ; select port J
$017 61 OP ; idle AUS code to U75 - no button pressed
$018 C1 LI #1 ; AC = 1
$019 84 LDZ #4 ; point at KEY2_UP
$01A 44 LHI #4 ; row 4
$01B 02 S ; KEY2_UP = 1
$01C 86 LDZ #6 ; point at KEY3_UP
$01D 44 LHI #4 ; row 4
$01E 02 S ; KEY3_UP = 1
$01F C8 LI #8 ; AC = 8
$020 86 LDZ #6 ; point at ATTSHADOW
$021 42 LHI #2 ; row 2
$022 02 S ; ATTSHADOW = 8
$023 86 LDZ #6 ; select port G
$024 61 OP ; idle the attenuator bus, _KBRESET released
$025 AC 00 CAL f_DefaultVolume ; choose the startup attenuation - $12 (a moderate level) if the main
$027 AF 7F CAL f_PowerSymbol ; set TMP1 to 12 (a bar glyph) when the main logic is down,
$029 C7 LI #7 ; AC = 7
$02A 03 TAE ; E = AC
$02B CD LI #13 ; timer period $7D - one display digit per tick
$02C F9 WTTM ; load the timer from E:AC
$02D 2C 81 SCTL $1 ; interrupts on: the panel lights up from here
$02F 68 3D JMP f_MainLoop ; keep the clock readout fresh continuously, and run everything else once every sixteen display ticks.
```

\$031-\$037 · 7 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

f_DefaultVolume \$400

logic is already up, \$22 (silence) if not - then push it to the attenuator.

Purpose choose the startup attenuation - \$12 (a moderate level) if the main

Reached from \$025

```
$400 89 LDZ #9 ; select port J
$401 0C IP ; AC = port J (AUS/MPS)
$402 89 LDZ #9 ; point at TMP0
$403 02 S ; TMP0 = AC
$404 C2 LI #2 ; mask bit 1
$405 E7 AND ; is the main logic powered?
$406 7E 12 BZ @skip ; if the masked bit is clear
$408 88 LDZ #8 ; point at ATTEN_LO
$409 43 LHI #3 ; row 3
$40A C2 LI #2 ; AC = 2
$40B 02 S ; powered: attenuation $12, a moderate level
$40C EE IND ; step to ATTEN_HI
$40D C1 LI #1 ; AC = 1
$40E 02 S ; ATTEN_HI = 1
$40F AD 46 CAL f_SetAttenuator ; push it to the attenuator
$411 62 RT ; return
@skip:
$412 88 LDZ #8 ; point at ATTEN_LO
$413 43 LHI #3 ; row 3
$414 C2 LI #2 ; AC = 2
$415 02 S ; unpowered: attenuation $22, silence
$416 EE IND ; step to ATTEN_HI
$417 C2 LI #2 ; AC = 2
$418 02 S ; ATTEN_HI = 2
$419 AD 46 CAL f_SetAttenuator ; push it to the attenuator
$41B 62 RT ; return
```

Interrupt vectors and main loop

v_TimerInterrupt \$038

\$038 68 98 **JMP** f_RefreshDigit ; light exactly one of the eight VFD digits, then set up for the next.

\$03A–\$03B · 2 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

v_SecondInterrupt \$03C

this firmware never enables that source.

\$03C 22 **RTI** ; return from the interrupt

f_MainLoop \$03D

TICK is incremented by the display ISR. While it is non-zero the loop only re-reads the clock and redraws the volume bar; when it wraps to zero the full service pass at \$046-\$056 runs, then \$058 waits for the next tick.

Purpose keep the clock readout fresh continuously, and run everything else once every sixteen display ticks.

Reached from \$02F, \$044, \$05D

```
$03D A9 6A CAL f_ReadClock ; read the time of day out of the real-time clock
$03F AA 78 CAL f_DrawVolumeBar ; redraw the volume bar
$041 8B LDZ #11 ; point at TICK
$042 40 LHI #0 ; row 0
$043 21 L ; the ISR bumps TICK every timer tick
$044 3E 3D BNZ f_MainLoop ; not yet wrapped: just keep the readout fresh and go round again
$046 AC 8A CAL f_ReadU75Command ; TICK wrapped - one service pass every 16 display ticks. Read CPCP from U75 first
$048 A9 04 CAL f_ScanKeys ; scan the front-panel key matrix
$04A AB 5F CAL f_LatchU75Requests ; edge-detect the U75 requests
$04C AB 74 CAL f_GenLockMode ; handle a genlock mode change
$04E AA 08 CAL f_PowerControl ; run the power on/off logic
$050 AE 00 CAL f_TransportButtons ; handle the CD transport buttons
$052 AB 00 CAL f_VolumeKeys ; handle volume up/down
$054 AA D8 CAL f_ReadGenLockMode ; read the Amiga genlock mode select lines
$056 A8 5F CAL f_KeyboardReset ; assert or release _KBRESET

@loop:
$058 8B LDZ #11 ; point at TICK
$059 40 LHI #0 ; row 0
$05A 21 L ; wait for the next tick so the service pass runs only once per wrap
$05B 7E 58 BZ @loop ; if it is zero
$05D 68 3D JMP f_MainLoop ; keep the clock readout fresh continuously, and run everything else once every sixteen display ticks.
```

f_RefreshDigit \$098

timer ISR: light one digit of the panel

Reads *DIGIT*, looks that digit's value up in the font, ORs in its bar segment from *BAR0-7*, drives *Sa-Sd* and *Se-Sh*, then strikes the matching grid. Both grid ports are blanked first so the previous digit cannot ghost onto the new one. Also runs the colon timer: while *COLON_ON* is set, *COLON_T_LO/HI* count 256 ticks and then clear it, which is what makes the colon blink.

Purpose light exactly one of the eight VFD digits, then set up for the next.
Timing reloads the timer with \$7D, so the panel is swept every eight ticks.
Reached from \$038

```
$098 E0    XA0                ; save the interrupted code's AC in A0
$099 0D    XAE                ; swap AC and E
$09A E4    XA1                ; save E in A1
$09B F0    XL0                ; save DPL in L0
$09C F8    XH0                ; save DPH in H0
$09D 8D    LDZ    #13         ; select port N
$09E C0    CLA                ; AC = 0
$09F 61    OP                ; blank G1-G4 before changing the segments
$0A0 EE    IND                ; step to port 0
$0A1 61    OP                ; blank G5-G8 too - stops the previous digit ghosting
$0A2 88    LDZ    #8          ; point at DIGIT
$0A3 40    LHI    #0          ; row 0
$0A4 21    L                ; which of the eight digits is due?
$0A5 40    LHI    #0          ; row 0
$0A6 F7    TAL                ; use it as a column index into the display buffer
$0A7 21    L                ; fetch this digit's font index
$0A8 03    TAE                ; E = AC
$0A9 80    LDZ    #0          ; set up E = 8: the font table lives at $080
$0AA 48    LHI    #8          ; row 8
$0AB 23    XAH                ; swap AC and the RAM row
$0AC 0D    XAE                ; swap AC and E
$0AD 63    RTBL               ; AC = Se-Sh, E = Sa-Sd for this digit
$0AE 8B    LDZ    #11         ; select port L
$0AF 0D    XAE                ; swap AC and E
$0B0 61    OP                ; light Sa-Sd for this digit
$0B1 0D    XAE                ; swap AC and E
$0B2 8B    LDZ    #11         ; point at SEGTMP
$0B3 43    LHI    #3          ; row 3
$0B4 02    S                ; hold Se-Sh while the extra bits are fetched
$0B5 88    LDZ    #8          ; point at DIGIT
$0B6 21    L                ; AC = DIGIT
$0B7 80    LDZ    #0          ; point at BAR0
$0B8 43    LHI    #3          ; row 3
$0B9 F7    TAL                ; DPL = AC - use it as an index
$0BA 21    L                ; this digit's volume-bar bit
$0BB 8B    LDZ    #11         ; point at SEGTMP
$0BC 43    LHI    #3          ; row 3
$0BD E5    OR                ; merge it into Se-Sh
$0BE 8C    LDZ    #12         ; select port M
$0BF 61    OP                ; light Se-Sh plus the indicator
$0C0 88    LDZ    #8          ; point at DIGIT
$0C1 40    LHI    #0          ; row 0
$0C2 21    L                ; AC = DIGIT
$0C3 03    TAE                ; E = AC
$0C4 80    LDZ    #0          ; set up E = 9: the grid table lives at $090
$0C5 49    LHI    #9          ; row 9
$0C6 23    XAH                ; swap AC and the RAM row
$0C7 0D    XAE                ; swap AC and E
$0C8 63    RTBL               ; look up which grid to strike for this digit
$0C9 8D    LDZ    #13         ; select port N
$0CA 61    OP                ; strike the grid, G1-G4 half
$0CB EE    IND                ; step to port 0
$0CC 0D    XAE                ; swap AC and E
$0CD 61    OP                ; and the G5-G8 half
$0CE 88    LDZ    #8          ; point at DIGIT
$0CF 40    LHI    #0          ; row 0
$0D0 21    L                ; AC = DIGIT
$0D1 0E    INC                ; move to the next digit
$0D2 02    S                ; DIGIT = AC
$0D3 C7    LI    #7          ; wrap 0-7
$0D4 E7    AND                ; AC = DIGIT AND %0111 -> keeps bits 0,1,2
$0D5 02    S                ; DIGIT = AC
$0D6 8F    LDZ    #15         ; point at COLON_ON
$0D7 21    L                ; is the colon timer running?
$0D8 7E E9 BZ    @skip        ; if it is zero
$0DA 8C    LDZ    #12         ; colon timer, low nibble
$0DB 21    L                ; AC = COLON_T_LO
$0DC 0E    INC                ; AC + 1
$0DD 02    S                ; COLON_T_LO = AC
$0DE 3E E9 BNZ    @skip        ; if not zero
$0E0 EE    IND                ; carry into the high nibble
$0E1 21    L                ; AC = COLON_T_HI
$0E2 0E    INC                ; AC + 1
$0E3 02    S                ; COLON_T_HI = AC
$0E4 3E E9 BNZ    @skip        ; if not zero
$0E6 8F    LDZ    #15         ; colon off
$0E7 C0    CLA                ; zero, for the store below
$0E8 02    S                ; 256 ticks up - colon off
```

@skip:

```
$0E9 8B LDZ #11 ; point at TICK
$0EA 40 LHI #0 ; row 0
$0EB 2E INM ; bump TICK - this is the main loop's 16-tick prescaler
$0EC C7 LI #7 ; AC = 7
$0ED 03 TAE ; E = AC
$0EE CD LI #13 ; AC = 13
$0EF F9 WTTM ; reload the timer for the next digit
$0F0 F8 XH0 ; restore DPH
$0F1 F0 XL0 ; restore DPL
$0F2 E4 XA1 ; restore E
$0F3 0D XAE ; swap AC and E
$0F4 E0 XA0 ; restore AC
$0F5 2C 81 SCTL $1 ; interrupts on
$0F7 22 RTI ; return from the interrupt
```

\$0F8–\$0FF · 8 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

Key strobe patterns \$100-\$103

Key matrix strobe patterns, fetched by RTBL at \$10A. Active low, one column at a time. Only the high nibble is used - RTBL discards the low nibble into E - so the stored \$EF \$DF \$BF \$7F become \$E \$D \$B \$7 on KST0-3.

Addr	Bytes	Meaning
\$100	\$EF	row 0: RTBL takes the high nibble \$E -> KST0 driven low, the rest released
\$101	\$DF	row 1: RTBL takes the high nibble \$D -> KST1 driven low, the rest released
\$102	\$BF	row 2: RTBL takes the high nibble \$B -> KST2 driven low, the rest released
\$103	\$7F	row 3: RTBL takes the high nibble \$7 -> KST3 driven low, the rest released

Front-panel keys

f_ScanKeys \$104

For each row it drives one KST line low, reads KI0-2 back, inverts so a pressed key reads as 1, and ORs in that row's VKEY cell - so an IR remote command is indistinguishable from a real press from here on. Each row has four state nibbles at KEY<row>_0..3:

```
_0 currently pressed
_1 went down this pass (new AND NOT previous - a rising edge)
_2 auto-repeat pulse
_3 hold counter
```

The hold counter runs while the key is down. When it reaches exactly 10 the repeat pulse fires; it then wraps round the nibble, so the pulse repeats every 16 passes after that - roughly 335 ms to the first repeat, then every ~540 ms. The routine loops back to itself until all four rows have been done.

Purpose strobe all four key columns, merge in the virtual keys U75 asked for, and work out which keys went down this pass.
Reached from \$048, \$15B

```
$104 80 LDZ #0 ; point at SCANROW
$105 44 LHI #4 ; row 4
$106 21 L ; AC = SCANROW
$107 0D XAE ; swap AC and E
$108 C0 CLA ; AC = 0
$109 0D XAE ; swap AC and E
$10A 63 RTBL ; active-low strobe pattern for this row
$10B 82 LDZ #2 ; select port C
$10C 61 OP ; drive it onto KST0-3
$10D A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$10F 21 L ; previous "pressed" state of this row
$110 89 LDZ #9 ; point at TMP0
$111 40 LHI #0 ; row 0
$112 02 S ; keep it
$113 CF LI #15 ; invert every bit
$114 F5 EXL ; TMP0 = NOT previous - the keys that were up
$115 02 S ; TMP0 = AC
$116 81 LDZ #1 ; select port B
$117 0C IP ; read KI0-2 back - a 0 means a key in this row is down
$118 A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$11A 02 S ; raw port B sample
$11B C3 LI #3 ; mask bits 0,1
$11C E7 AND ; keep KI0 and KI1 only - KI2 is discarded
$11D 02 S ; KEY_0 = AC
$11E C3 LI #3 ; toggle bits 0,1
$11F F5 EXL ; invert: 1 = pressed
$120 02 S ; KEY_0 = currently pressed
$121 80 LDZ #0 ; point at SCANROW
$122 44 LHI #4 ; row 4
$123 21 L ; which row are we scanning?
$124 0D XAE ; swap AC and E
$125 80 LDZ #0 ; point at VKEY0
$126 45 LHI #5 ; row 5
$127 0D XAE ; swap AC and E
$128 F7 TAL ; index RAM row 5 by it
$129 21 L ; VKEY - the virtual keypress U75 asked for
$12A A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$12C E5 OR ; merge the U75 virtual key into this row's real key bits
$12D 02 S ; KEY_0 = pressed, real or virtual
$12E 89 LDZ #9 ; point at TMP0
$12F 40 LHI #0 ; row 0
$130 21 L ; the keys that were up last pass
$131 A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$133 E7 AND ; pressed now AND up before = went down this pass
$134 EE IND ; step to KEY_1
$135 02 S ; KEY_1 = press edges
$136 C0 CLA ; AC = 0
$137 EE IND ; step to KEY_2
$138 02 S ; KEY_2 = 0, clear the repeat pulse
$139 A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$13B 21 L ; anything held in this row?
$13C 7E 4E BZ @skip ; no - reset the hold counter
$13E EE IND ; step to KEY_1
$13F EE IND ; step to KEY_2
$140 EE IND ; step to KEY_3
$141 2E INM ; hold counter + 1
$142 21 L ; AC = KEY_3
$143 2C 4A CI #10 ; exactly 10?
$145 3E 55 BNZ @skip2 ; not yet, or already past - no pulse
$147 A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$149 21 L ; held keys
$14A EE IND ; step to KEY_1
$14B E5 OR ; OR the press edges in
$14C EE IND ; step to KEY_2
$14D 02 S ; KEY_2 = repeat pulse for those keys
@skip:
$14E A9 5E CAL f_KeyRowPtr ; dP -> this row's KEY block, column 0.
$150 EE IND ; step to KEY_1
$151 EE IND ; step to KEY_2
$152 EE IND ; step to KEY_3
$153 C0 CLA ; zero, for the store below
$154 02 S ; KEY_3 = 0
@skip2:
$155 80 LDZ #0 ; point at SCANROW
```

```

$156 44 LHI #4 ; row 4
$157 2E INM ; next row
$158 C3 LI #3 ; mask bits 0,1
$159 E7 AND ; wrap 0-3
$15A 02 S ; SCANROW = AC
$15B 3E 04 BNZ f_ScanKeys ; repeat until all four rows are strobed
$15D 62 RT ; return

```

f_KeyRowPtr \$15E

CLC/RAL/RAL multiplies SCANROW by four; DPH is set to 1, so row r lives at \$1(4r)..\$1(4r+3). AC is preserved through working register A2.

Purpose DP -> this row's KEY block, column 0.

Reached from \$10D, \$118, \$12A, \$131, \$139, \$147, \$14E

```

$15E E8 XA2 ; park the caller's AC in A2
$15F 80 LDZ #0 ; point at SCANROW
$160 44 LHI #4 ; row 4
$161 E1 CLC ; clear carry before the shift
$162 21 L ; AC = SCANROW
$163 01 RAL ; row * 2
$164 01 RAL ; row * 4 - four state nibbles per row
$165 80 LDZ #0 ; point at KEY0_0
$166 41 LHI #1 ; row 1
$167 F7 TAL ; point DP at this row's block in RAM row 1
$168 E8 XA2 ; bring the caller's AC back
$169 62 RT ; return

```

f_ArmRepeat \$78E

start a hold burst: RPT_CMD = which key (1 or 2), RPT_TICK = 0, RPT_DELAY = 2 more issues to go.

Reached from \$6AB, \$6C2

```

$78E 8A LDZ #10 ; point at RPT_CMD
$78F 45 LHI #5 ; row 5
$790 02 S ; remember which button is repeating
$791 8B LDZ #11 ; point at RPT_TICK
$792 45 LHI #5 ; row 5
$793 C0 CLA ; zero, for the store below
$794 02 S ; RPT_TICK = 0
$795 EE IND ; step to RPT_DELAY
$796 C2 LI #2 ; two more issues after this one
$797 02 S ; RPT_DELAY = 2
$798 62 RT ; return

```

f_RepeatTick \$799

burst is armed it counts to three, re-issues the hold command, and lets f_RepeatTimeout decide whether that was the last one.

Purpose deliver the rest of a hold burst. Called every service pass; while a

Reached from \$607

```

$799 8A LDZ #10 ; point at RPT_CMD
$79A 45 LHI #5 ; row 5
$79B 21 L ; is a burst in progress?
$79C 7E B4 BZ @done ; if it is zero
$79E 8B LDZ #11 ; point at RPT_TICK
$79F 45 LHI #5 ; row 5
$7A0 21 L ; AC = RPT_TICK
$7A1 0E INC ; count the ticks
$7A2 02 S ; RPT_TICK = AC
$7A3 2C 43 CI #3 ; every third pass
$7A5 3E B4 BNZ @done ; if AC != 3
$7A7 C0 CLA ; zero, for the store below
$7A8 02 S ; RPT_TICK = 0
$7A9 8A LDZ #10 ; point at RPT_CMD
$7AA 45 LHI #5 ; row 5
$7AB 21 L ; AC = RPT_CMD
$7AC 2C 41 CI #1 ; key 2?
$7AE 7E B5 BZ @skip ; if AC == 1
$7B0 2C 42 CI #2 ; key 3?
$7B2 7E BA BZ @skip2 ; if AC == 2
@done:
$7B4 62 RT ; return
@skip:
$7B5 AE AD CAL f_SkipBack ; re-issue the hold command
$7B7 AF BF CAL f_RepeatTimeout ; one fewer issue left in the burst; at zero, disarm it
$7B9 62 RT ; return
@skip2:
$7BA AE C4 CAL f_SkipForward ; re-issue the hold command
$7BC AF BF CAL f_RepeatTimeout ; one fewer issue left in the burst; at zero, disarm it
$7BE 62 RT ; return

```

one fewer issue left in the burst; at zero, disarm it.

Reached from \$7B7, \$7BC

```
$7BF 8C    LDZ    #12          ; point at RPT_DELAY
$7C0 45    LHI    #5          ; row 5
$7C1 21    L      ; AC = RPT_DELAY
$7C2 0F    DEC          ; one fewer to go
$7C3 02    S      ; RPT_DELAY = AC
$7C4 3E CA  BNZ    @done      ; if not zero
$7C6 8A    LDZ    #10          ; point at RPT_CMD
$7C7 45    LHI    #5          ; row 5
$7C8 C0    CLA          ; zero, for the store below
$7C9 02    S      ; burst finished - disarm
@done:
$7CA 62    RT          ; return
```

Real-time clock and the time display

f_ReadClock \$16A

U61 is an OKI MSM6242B real-time clock. Port E carries its register address, port D its data, and PP0 through U80 is the read strobe. The Amiga reaches the same chip through the U57/U59 buffers, so the clock is shared. Registers 5, 4, 3, 2 - hours tens, hours units, minutes tens, minutes units - go straight into HRS10, HRS1, MIN10, MIN1. HRS10 is masked to two bits because the clock runs in 24-hour mode, so that digit is only ever 0, 1 or 2; a leading zero shows as blank. AMPM: control register F bit 2 is the 24/12-hour select. In 24-hour mode the symbol is blank. In 12-hour mode - which only the Amiga could select - it shows one bar for AM and the other for PM, taken from bit 2 of the hours-tens register. COLON: register 0 is the seconds units. When it differs from SEC_PREV the colon is switched on and COLON_ON armed; the ISR clears it 256 ticks (~0.5 s) later, so the colon flashes once a second.

Purpose read the time of day out of the real-time clock into the display buffer, and set the AM/PM and colon symbols.

Reached from \$03D

```
$16A 84 LDZ #4 ; select port E
$16B C5 LI #5 ; value 5 = %0101
$16C 61 OP ; RTC register 5 - tens of hours
$16D AA 00 CAL f_ClockReadHigh ; raise the clock read strobe
$16F 83 LDZ #3 ; select port D
$170 0C IP ; read the digit back
$171 89 LDZ #9 ; point at TMP0
$172 02 S ; TMP0 = AC
$173 C3 LI #3 ; mask bits 0,1
$174 E7 AND ; 24-hour mode, so this digit is only ever 0, 1 or 2
$175 02 S ; TMP0 = AC
$176 3E 7A BNZ @skip ; if not zero
$178 CA LI #10 ; a leading zero shows as blank
$179 02 S ; TMP0 = 10
@skip:
$17A 21 L ; AC = the digit, or blank
$17B 80 LDZ #0 ; point at HRS10
$17C 02 S ; tens of hours
$17D AA 04 CAL f_ClockReadLow ; drop the read strobe
$17F 84 LDZ #4 ; select port E
$180 C4 LI #4 ; value 4 = %0100
$181 61 OP ; RTC register 4 - units of hours
$182 AA 00 CAL f_ClockReadHigh ; raise the RTC read strobe (PP0 -> U80 pin 2)
$184 83 LDZ #3 ; select port D
$185 0C IP ; AC = port D (RTC data)
$186 81 LDZ #1 ; point at HRS1
$187 02 S ; units of hours
$188 AA 04 CAL f_ClockReadLow ; drop the RTC read strobe
$18A 84 LDZ #4 ; select port E
$18B C3 LI #3 ; value 3 = %0011
$18C 61 OP ; RTC register 3 - tens of minutes
$18D AA 00 CAL f_ClockReadHigh ; raise the RTC read strobe (PP0 -> U80 pin 2)
$18F 83 LDZ #3 ; select port D
$190 0C IP ; AC = port D (RTC data)
$191 82 LDZ #2 ; point at MIN10
$192 02 S ; tens of minutes
$193 AA 04 CAL f_ClockReadLow ; drop the RTC read strobe
$195 84 LDZ #4 ; select port E
$196 C2 LI #2 ; value 2 = %0010
$197 61 OP ; RTC register 2 - units of minutes
$198 AA 00 CAL f_ClockReadHigh ; raise the RTC read strobe (PP0 -> U80 pin 2)
$19A 83 LDZ #3 ; select port D
$19B 0C IP ; AC = port D (RTC data)
$19C 83 LDZ #3 ; point at MIN1
$19D 02 S ; units of minutes
$19E AA 04 CAL f_ClockReadLow ; drop the RTC read strobe
$1A0 84 LDZ #4 ; select port E
$1A1 C0 CLA ; AC = 0
$1A2 61 OP ; RTC register 0 - units of seconds, used to spot the tick
$1A3 AA 00 CAL f_ClockReadHigh ; raise the RTC read strobe (PP0 -> U80 pin 2)
$1A5 83 LDZ #3 ; select port D
$1A6 0C IP ; AC = port D (RTC data)
$1A7 88 LDZ #8 ; point at SEC_NOW
$1A8 44 LHI #4 ; row 4
$1A9 02 S ; SEC_NOW = AC
$1AA AA 04 CAL f_ClockReadLow ; drop the RTC read strobe
$1AC 8A LDZ #10 ; point at TMP1
$1AD CA LI #10 ; assume 24-hour mode: AM/PM symbol blank
$1AE 02 S ; TMP1 = 10
$1AF 84 LDZ #4 ; select port E
$1B0 CF LI #15 ; value 15 = %1111
$1B1 61 OP ; RTC control register F
$1B2 AA 00 CAL f_ClockReadHigh ; raise the RTC read strobe (PP0 -> U80 pin 2)
$1B4 83 LDZ #3 ; select port D
$1B5 0C IP ; AC = port D (RTC data)
$1B6 89 LDZ #9 ; point at TMP0
$1B7 02 S ; TMP0 = AC
$1B8 C4 LI #4 ; mask bit 2
$1B9 E7 AND ; bit 2 = 24-hour mode selected?
$1BA 3E D1 BNZ @skip2 ; 24-hour: leave AM/PM blank
$1BC 8A LDZ #10 ; point at TMP1
$1BD CB LI #11 ; 12-hour mode: AM symbol
$1BE 02 S ; TMP1 = 11
$1BF AA 04 CAL f_ClockReadLow ; drop the RTC read strobe
$1C1 84 LDZ #4 ; select port E
$1C2 C5 LI #5 ; RTC register 5 - hours tens, whose bit 2 is the PM flag
$1C3 61 OP ; port E (RTC address) = AC
```

```

$1C4 AA 00 CAL f_ClockReadHigh ; raise the RTC read strobe (PP0 -> U80 pin 2)
$1C6 83 LDZ #3 ; select port D
$1C7 0C IP ; AC = port D (RTC data)
$1C8 89 LDZ #9 ; point at TMP0
$1C9 02 S ; TMP0 = AC
$1CA C4 LI #4 ; mask bit 2
$1CB E7 AND ; PM?
$1CC 7E D1 BZ @skip2 ; if the masked bit is clear
$1CE 8A LDZ #10 ; point at TMP1
$1CF CC LI #12 ; PM symbol
$1D0 02 S ; TMP1 = 12

@skip2:
$1D1 8A LDZ #10 ; point at TMP1
$1D2 21 L ; AC = TMP1
$1D3 86 LDZ #6 ; point at AMPM
$1D4 02 S ; write the AM/PM symbol
$1D5 AA 04 CAL f_ClockReadLow ; drop the RTC read strobe
$1D7 8F LDZ #15 ; point at COLON_ON
$1D8 21 L ; AC = COLON_ON
$1D9 3E FA BNZ @skip4 ; colon still lit from the last tick?
$1DB 89 LDZ #9 ; point at TMP0
$1DC CA LI #10 ; default: colon off
$1DD 02 S ; TMP0 = 10
$1DE 88 LDZ #8 ; point at SEC_NOW
$1DF 44 LHI #4 ; row 4
$1E0 21 L ; AC = SEC_NOW
$1E1 8E LDZ #14 ; point at SEC_PREV
$1E2 FB CM ; has the seconds digit changed?
$1E3 7E F0 BZ @skip3 ; no - leave the colon off
$1E5 89 LDZ #9 ; point at TMP0
$1E6 CD LI #13 ; yes: colon on (both bars)
$1E7 02 S ; TMP0 = 13
$1E8 8C LDZ #12 ; point at COLON_T_LO
$1E9 C0 CLA ; restart the 256-tick colon timer
$1EA 02 S ; COLON_T_LO = 0
$1EB EE IND ; step to COLON_T_HI
$1EC 02 S ; COLON_T_HI = 0
$1ED 0E INC ; and arm it
$1EE 8F LDZ #15 ; point at COLON_ON
$1EF 02 S ; COLON_ON = AC

@skip3:
$1F0 89 LDZ #9 ; point at TMP0
$1F1 21 L ; AC = TMP0
$1F2 87 LDZ #7 ; point at COLON
$1F3 02 S ; write the colon
$1F4 88 LDZ #8 ; point at SEC_NOW
$1F5 44 LHI #4 ; row 4
$1F6 21 L ; AC = SEC_NOW
$1F7 8E LDZ #14 ; point at SEC_PREV
$1F8 02 S ; remember the seconds digit for next pass
$1F9 62 RT ; return

@skip4:
$1FA 87 LDZ #7 ; point at COLON
$1FB CD LI #13 ; colon on
$1FC 02 S ; COLON = 13
$1FD 62 RT ; return

```

\$1FE–\$1FF · 2 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

f_ClockReadHigh \$200

raise the RTC read strobe (PP0 -> U80 pin 2).

Reached from \$16D, \$182, \$18D, \$198, \$1A3, \$1B2, \$1C4

```

$200 8F LDZ #15 ; select port P
$201 C1 LI #1 ; value 1 = %0001
$202 61 OP ; raise the RTC read strobe (PP0 -> U80)
$203 62 RT ; return

```

f_ClockReadLow \$204

drop the RTC read strobe. Paired with \$200 around every register read from the clock.

Reached from \$17D, \$188, \$193, \$19E, \$1AA, \$1BF, \$1D5

```

$204 8F LDZ #15 ; select port P
$205 C0 CLA ; AC = 0
$206 61 OP ; drop the RTC read strobe
$207 62 RT ; return

```

f_ClockWrite \$2CE

pulse the RTC write strobe (PI2 -> U74 pin 6). Low-high-low. MS0/MS1 and AUPLY are left released throughout.

Reached from \$3BB, \$3C3, \$3CE, \$3DE, \$3E6

```
$2CE CB      LI      #11          ; AC = 11
$2CF 88      LDZ     #8           ; select port I
$2D0 61      OP           ; RTC write strobe low
$2D1 CF      LI      #15          ; AC = 15
$2D2 88      LDZ     #8           ; select port I
$2D3 61      OP           ; high
$2D4 CB      LI      #11          ; AC = 11
$2D5 88      LDZ     #8           ; select port I
$2D6 61      OP           ; low again - one write pulse to the clock
$2D7 62      RT           ; return
```

f_InitRTC \$3B2

A textbook MSM6242 initialisation. Control register F is written \$7 (RESET, STOP and 24-hour all set), then \$6 (reset released, still stopped), the six time registers 0-5 are cleared, control register D is zeroed, and finally F is written \$4 - STOP released, 24-hour mode - which starts the clock. That last write is why HRS10 only ever needs two bits.

Purpose bring up the MSM6242B real-time clock at power-on.

Reached from \$010

```
$3B2 8F      LDZ     #15          ; select port P
$3B3 C0      CLA           ; AC = 0
$3B4 61      OP           ; RTC read strobe idle
$3B5 84      LDZ     #4           ; select port E
$3B6 CF      LI      #15          ; value 15 = %1111
$3B7 61      OP           ; RTC control register F
$3B8 83      LDZ     #3           ; select port D
$3B9 C7      LI      #7           ; value 7 = %0111
$3BA 61      OP           ; $7 - assert RESET and STOP, select 24-hour mode
$3BB AA CE   CAL     f_ClockWrite ; pulse the clock write strobe
$3BD 84      LDZ     #4           ; select port E
$3BE CF      LI      #15          ; value 15 = %1111
$3BF 61      OP           ; control register F again
$3C0 83      LDZ     #3           ; select port D
$3C1 C6      LI      #6           ; value 6 = %0110
$3C2 61      OP           ; $6 - release RESET, keep STOP and 24-hour
$3C3 AA CE   CAL     f_ClockWrite ; pulse the RTC write strobe (PI2 -> U74 pin 6)
$3C5 89      LDZ     #9           ; point at TMP0
$3C6 C0      CLA           ; zero, for the store below
$3C7 02      S           ; TMP0 = 0
$3C8 83      LDZ     #3           ; select port D
$3C9 61      OP           ; data = 0
@loop:
$3CA 89      LDZ     #9           ; point at TMP0
$3CB 21      L           ; AC = TMP0
$3CC 84      LDZ     #4           ; select port E
$3CD 61      OP           ; clear time register n
$3CE AA CE   CAL     f_ClockWrite ; pulse the RTC write strobe (PI2 -> U74 pin 6)
$3D0 89      LDZ     #9           ; point at TMP0
$3D1 21      L           ; AC = TMP0
$3D2 0E      INC          ; AC + 1
$3D3 02      S           ; TMP0 = AC
$3D4 2C 46   CI      #6           ; is AC 6?
$3D6 3E CA   BNZ     @loop        ; registers 0 to 5
$3D8 84      LDZ     #4           ; select port E
$3D9 CD      LI      #13          ; value 13 = %1101
$3DA 61      OP           ; control register D, cleared
$3DB 83      LDZ     #3           ; select port D
$3DC C0      CLA           ; AC = 0
$3DD 61      OP           ; data 0
$3DE AA CE   CAL     f_ClockWrite ; pulse the RTC write strobe (PI2 -> U74 pin 6)
$3E0 84      LDZ     #4           ; select port E
$3E1 CF      LI      #15          ; value 15 = %1111
$3E2 61      OP           ; control register F
$3E3 83      LDZ     #3           ; select port D
$3E4 C4      LI      #4           ; value 4 = %0100
$3E5 61      OP           ; $4 - release STOP, 24-hour mode: the clock starts here
$3E6 AA CE   CAL     f_ClockWrite ; pulse the RTC write strobe (PI2 -> U74 pin 6)
$3E8 83      LDZ     #3           ; select port D
$3E9 CF      LI      #15          ; value 15 = %1111
$3EA 61      OP           ; release the clock data lines
$3EB 62      RT           ; return
```

\$3EC–\$3EF · 4 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

\$3F0–\$3FF · 16 bytes · Unprogrammed EPROM. \$FF decodes as XD, but nothing reaches it.

CD drive

f_SendCDCCommand \$41C

The tail is clear: POLL_TICK is reused as a counter, set to 8, and the loop at \$43E toggles PF2 (SCK) between values 3 and 7 eight times with both data lines released - eight clock pulses at ~71 kHz. The setup uses four register addresses reachable only behind a BANK prefix: reg12 <- 1, reg14 <- CDCMD_LO, reg15 <- CDCMD_HI, then reg13 pulsed 8,9,8. These are almost certainly the LC6554's own serial-interface registers: the datasheet shows a serial shift register and mode registers but gives no instruction for loading them, and the data lines being released during the burst is exactly what you do when internal hardware owns SI/SO. See BANK-conundrum.md for the full evidence. It is not settled.

Purpose clock the pending CD command out on SDATA/SCK.

Reached from \$5DA, \$6F7

```
$41C 85    LDZ    #5           ; select port F
$41D C6    LI     #6           ; value 6 = %0110
$41E 61    OP           ; SDATA pulled low (PF0=0), SCK high - a start marker
$41F C1    LI     #1           ; AC = 1
$420 8C    LDZ    #12          ; select port M
$421 FD    BANK         ; BANK - see the note at $41C
$422 61    OP           ; write reg 12 through the BANK prefix - see $41C
$423 81    LDZ    #1           ; point at CDCMD_LO
$424 44    LHI    #4           ; row 4
$425 21    L           ; AC = CDCMD_LO
$426 8E    LDZ    #14          ; select port 0
$427 FD    BANK         ; BANK - see the note at $41C
$428 61    OP           ; write reg 14 through the BANK prefix - see $41C
$429 82    LDZ    #2           ; point at CDCMD_HI
$42A 44    LHI    #4           ; row 4
$42B 21    L           ; AC = CDCMD_HI
$42C 8F    LDZ    #15          ; select port P
$42D FD    BANK         ; BANK - see the note at $41C
$42E 61    OP           ; write reg 15 through the BANK prefix - see $41C
$42F C8    LI     #8           ; AC = 8
$430 8D    LDZ    #13          ; select port N
$431 FD    BANK         ; BANK - see the note at $41C
$432 61    OP           ; write reg 13 through the BANK prefix - see $41C
$433 C9    LI     #9           ; AC = 9
$434 8D    LDZ    #13          ; select port N
$435 FD    BANK         ; BANK - see the note at $41C
$436 61    OP           ; write reg 13 through the BANK prefix - see $41C
$437 C8    LI     #8           ; AC = 8
$438 8D    LDZ    #13          ; select port N
$439 FD    BANK         ; BANK - see the note at $41C
$43A 61    OP           ; write reg 13 through the BANK prefix - see $41C
$43B 89    LDZ    #9           ; point at TMP0
$43C C8    LI     #8           ; AC = 8
$43D 02    S           ; eight bits to clock (POLL_TICK reused as the counter)

@loop:
$43E 85    LDZ    #5           ; select port F
$43F C3    LI     #3           ; value 3 = %0011
$440 61    OP           ; SCK low
$441 00    NOP          ; timing pad
$442 00    NOP          ; timing pad
$443 85    LDZ    #5           ; select port F
$444 C7    LI     #7           ; value 7 = %0111
$445 61    OP           ; SCK high
$446 00    NOP          ; timing pad
$447 00    NOP          ; timing pad
$448 89    LDZ    #9           ; point at TMP0
$449 2F    DEM         ; TMP0 -= 1
$44A 3E 3E BNZ    @loop       ; next bit
$44C C0    CLA         ; AC = 0
$44D 8D    LDZ    #13          ; select port N
$44E FD    BANK         ; BANK - see the note at $41C
$44F 61    OP           ; write reg 13 through the BANK prefix - see $41C
$450 62    RT          ; return
```

f_SendCDCCommand2 \$451

second phase of the CD exchange, eight more SCK pulses. reg12 <- 1, reg13 pulsed 0,1,0, then the burst with SCK left high. Under the serial-register reading this is the receive half; \$7CB then reads the answer out of reg14/reg15.

Reached from \$6FB

```
$451 85    LDZ    #5           ; select port F
$452 C7    LI     #7           ; value 7 = %0111
$453 61    OP           ; SDATA released, SCK high
$454 C1    LI     #1           ; AC = 1
$455 8C    LDZ    #12          ; select port M
$456 FD    BANK         ; BANK - see the note at $41C
$457 61    OP           ; write reg 12 through the BANK prefix - see $41C
$458 C0    CLA         ; AC = 0
$459 8D    LDZ    #13          ; select port N
$45A FD    BANK         ; BANK - see the note at $41C
$45B 61    OP           ; write reg 13 through the BANK prefix - see $41C
$45C C1    LI     #1           ; AC = 1
$45D 8D    LDZ    #13          ; select port N
$45E FD    BANK         ; BANK - see the note at $41C
$45F 61    OP           ; write reg 13 through the BANK prefix - see $41C
$460 C0    CLA         ; AC = 0
```

```

$461 8D      LDZ    #13          ; select port N
$462 FD      BANK          ; BANK - see the note at $41C
$463 61      OP          ; write reg 13 through the BANK prefix - see $41C
$464 89      LDZ    #9          ; point at TMP0
$465 C8      LI      #8          ; AC = 8
$466 02      S          ; TMP0 = 8

@loop:
$467 85      LDZ    #5          ; select port F
$468 C7      LI      #7          ; value 7 = %0111
$469 61      OP          ; SCK high
$46A 00      NOP          ; timing pad
$46B 00      NOP          ; timing pad
$46C 85      LDZ    #5          ; select port F
$46D C3      LI      #3          ; value 3 = %0011
$46E 61      OP          ; SCK low
$46F 00      NOP          ; timing pad
$470 00      NOP          ; timing pad
$471 89      LDZ    #9          ; point at TMP0
$472 2F      DEM          ; TMP0 -= 1
$473 3E 67   BNZ    @loop       ; if not yet 0
$475 85      LDZ    #5          ; select port F
$476 C7      LI      #7          ; value 7 = %0111
$477 61      OP          ; leave SCK high
$478 62      RT          ; return

```

f_Wait \$479

a nested busy-wait of about 4.2 ms. Two passes of the loop at \$47B, roughly 4096 x 7 instructions. The five NOPs in the inner loop are there purely to stretch it.

Reached from \$6F9

```

$479 AC 7B   CAL    @skip       ; run the delay loop twice: once by this call, once by falling into it
@skip:
$47B 8A      LDZ    #10          ; point at TMP1
$47C C0      CLA          ; zero, for the store below
$47D 02      S          ; TMP1 = 0

@loop:
$47E 0F      DEC          ; AC - 1
$47F 00      NOP          ; timing pad
$480 00      NOP          ; timing pad
$481 00      NOP          ; timing pad
$482 00      NOP          ; timing pad
$483 00      NOP          ; timing pad
$484 3E 7E   BNZ    @loop       ; if not zero
$486 2F      DEM          ; TMP1 -= 1: 16 inner loops per outer
$487 3E 7E   BNZ    @loop       ; if not yet 0
$489 62      RT          ; return

```

f_TransportButtons \$600

Six actions from four physical keys. Keys 0 and 1 fire on the press edge. Keys 2 and 3 each do two things (from the panel; the IR remote can only tap them, see f_ReadU75Command): a short press fires the tap command when the key is released (only if it was held fewer than 10 passes and the hold action has not fired), and holding it fires the hold command as a burst of three - once when the scanner's repeat pulse arrives, then twice more three passes apart - repeated every 16 passes for as long as the key stays down.

key	CD cmd	AUS	U75	scancode	when
row 0 bit 0	\$01	10	\$73		press edge, after an AUPLY pulse
row 1 bit 0	\$06	6	\$72		press edge
row 2 bit 0	\$02	3	\$75		released before 10 passes
row 2 bit 0	\$03	11	\$77		held: burst of three
row 3 bit 0	\$04	14	\$74		released before 10 passes
row 3 bit 0	\$05	7	\$76		held: burst of three

When nothing is pressed, POLL_TICK counts passes and every tenth one sends \$10, the track-number query, instead.

Purpose turn front-panel presses into CD commands and AUS event codes for U75.

Reached from \$050

```

$600 89      LDZ    #9          ; point at PORTJ
$601 42      LHI    #2          ; row 2
$602 C2      LI      #2          ; mask bit 1
$603 E7      AND          ; AC = PORTJ AND %0010 -> keeps bit 1
$604 02      S          ; PORTJ shadow = MPS bit only, AUS lines 0
$605 89      LDZ    #9          ; select port J
$606 61      OP          ; idle the AUS lines
$607 AF 99   CAL    f_RepeatTick ; service the auto-repeat timer
$609 81      LDZ    #1          ; point at KEY0_1
$60A 41      LHI    #1          ; row 1
$60B C1      LI      #1          ; mask bit 0
$60C E7      AND          ; AC = KEY0_1 AND %0001 -> keeps bit 0
$60D 7E 2F   BZ     @skip       ; play not pressed
$60F 83      LDZ    #3          ; point at CDCMD_X
$610 44      LHI    #4          ; row 4
$611 02      S          ; CDCMD_X = AC
$612 88      LDZ    #8          ; select port I
$613 CB      LI      #11         ; value 11 = %1011
$614 61      OP          ; port I: AUPLY high
$615 88      LDZ    #8          ; select port I
$616 C3      LI      #3          ; value 3 = %0011

```

```

$617 61      OP      ; pull AUPLY (CD Audio Play) low - PI3 is the only bit that changes
$618 89      LDZ     #9      ; point at TMP0
$619 C0      CLA     ; zero, for the store below
$61A 02      S      ; TMP0 = 0
$61B CC      LI      #12     ; AC = 12

@loop:
$61C 2E      INM      ; hold it low for ~4096 ticks
$61D 3E 1C   BNZ     @loop   ; if it has not wrapped yet
$61F 0E      INC      ; AC + 1
$620 3E 1C   BNZ     @loop   ; if it has not wrapped yet
$622 88      LDZ     #8      ; select port I
$623 CB      LI      #11     ; value 11 = %1011
$624 61      OP      ; release AUPLY again
$625 81      LDZ     #1      ; point at CDCMD_LO
$626 44      LHI     #4      ; row 4
$627 C1      LI      #1      ; AC = 1
$628 02      S      ; CD command 1
$629 EE      IND      ; step to CDCMD_HI
$62A C0      CLA     ; zero, for the store below
$62B 02      S      ; CDCMD_HI = 0
$62C CA      LI      #10     ; AUS code 10 -> U75 scancode $73, Play/Pause
$62D 6D D5   JMP     f_IssueButton ; report a front-panel button press to both sides of the machine.

@skip:
$62F 85      LDZ     #5      ; point at KEY1_1
$630 41      LHI     #1      ; row 1
$631 C1      LI      #1      ; mask bit 0
$632 E7      AND      ; AC = KEY1_1 AND %0001 -> keeps bit 0
$633 7E 42   BZ      @skip2  ; button 1 not pressed
$635 81      LDZ     #1      ; point at CDCMD_LO
$636 44      LHI     #4      ; row 4
$637 C6      LI      #6      ; CD command 6
$638 02      S      ; CDCMD_LO = 6
$639 EE      IND      ; step to CDCMD_HI
$63A C0      CLA     ; zero, for the store below
$63B 02      S      ; CDCMD_HI = 0
$63C 83      LDZ     #3      ; point at CDCMD_X
$63D 44      LHI     #4      ; row 4
$63E 02      S      ; CDCMD_X = 0
$63F C6      LI      #6      ; AUS code 6 -> U75 scancode $72, Stop
$640 6D D5   JMP     f_IssueButton ; report a front-panel button press to both sides of the machine.

@skip2:
$642 84      LDZ     #4      ; point at KEY2_UP
$643 44      LHI     #4      ; row 4
$644 21      L      ; KEY2_UP holds NOT-down from last pass
$645 89      LDZ     #9      ; point at TMP0
$646 02      S      ; TMP0 = AC
$647 C1      LI      #1      ; toggle bit 0
$648 F5      EXL      ; so this is: was it down last pass?
$649 02      S      ; TMP0 = AC
$64A 88      LDZ     #8      ; point at KEY2_0
$64B 41      LHI     #1      ; row 1
$64C 21      L      ; is key 2 down now?
$64D 8A      LDZ     #10     ; point at TMP1
$64E 02      S      ; TMP1 = AC
$64F C1      LI      #1      ; mask bit 0
$650 E7      AND      ; bit 0 only
$651 02      S      ; TMP1 = AC
$652 C1      LI      #1      ; toggle bit 0
$653 F5      EXL      ; invert: 1 = up
$654 84      LDZ     #4      ; point at KEY2_UP
$655 44      LHI     #4      ; row 4
$656 02      S      ; KEY2_UP = up now, kept for next pass
$657 89      LDZ     #9      ; point at TMP0
$658 21      L      ; down last pass
$659 84      LDZ     #4      ; point at KEY2_UP
$65A 44      LHI     #4      ; row 4
$65B E7      AND      ; AND up now = just released
$65C 7E 73   BZ      @skip3  ; not a release - nothing to do for key 2
$65E EE      IND      ; step to HOLD2_FIRED
$65F 21      L      ; did the hold action already fire?
$660 3E 73   BNZ     @skip3  ; yes - this release is not a tap
$662 8B      LDZ     #11     ; point at KEY2_3
$663 41      LHI     #1      ; row 1
$664 21      L      ; how long was it held?
$665 2C 4A   CI      #10     ; 10 passes or more?
$667 7F 73   BC      @skip3  ; that was a hold, not a tap
$669 81      LDZ     #1      ; point at CDCMD_LO
$66A 44      LHI     #4      ; row 4
$66B C2      LI      #2      ; tap: CD command $02
$66C 02      S      ; CDCMD_LO = 2
$66D EE      IND      ; step to CDCMD_HI
$66E C0      CLA     ; zero, for the store below
$66F 02      S      ; CDCMD_HI = 0
$670 C3      LI      #3      ; AUS code 3 -> U75 scancode $75
$671 6D D5   JMP     f_IssueButton ; report a front-panel button press to both sides of the machine.

@skip3:
$673 86      LDZ     #6      ; point at KEY3_UP
$674 44      LHI     #4      ; row 4
$675 21      L      ; KEY3_UP holds NOT-down from last pass
$676 89      LDZ     #9      ; point at TMP0
$677 02      S      ; TMP0 = AC
$678 C1      LI      #1      ; toggle bit 0
$679 F5      EXL      ; so this is: was it down last pass?
$67A 02      S      ; TMP0 = AC
$67B 8C      LDZ     #12     ; point at KEY3_0
$67C 41      LHI     #1      ; row 1

```

```

$67D 21      L           ; is key 3 down now?
$67E 8A      LDZ    #10   ; point at TMP1
$67F 02      S           ; TMP1 = AC
$680 C1      LI     #1    ; mask bit 0
$681 E7      AND     ; bit 0 only
$682 02      S           ; TMP1 = AC
$683 C1      LI     #1    ; toggle bit 0
$684 F5      EXL       ; invert: 1 = up
$685 86      LDZ    #6    ; point at KEY3_UP
$686 44      LHI     #4    ; row 4
$687 02      S           ; KEY3_UP = up now, kept for next pass
$688 89      LDZ    #9    ; point at TMP0
$689 21      L           ; down last pass
$68A 86      LDZ    #6    ; point at KEY3_UP
$68B 44      LHI     #4    ; row 4
$68C E7      AND     ; AND up now = just released
$68D 7E A4   BZ     @skip4 ; not a release - nothing to do for key 3
$68F EE      IND     ; step to HOLD3_FIRED
$690 21      L           ; did the hold action already fire?
$691 3E A4   BNZ    @skip4 ; yes - this release is not a tap
$693 8F      LDZ    #15   ; point at KEY3_3
$694 41      LHI     #1    ; row 1
$695 21      L           ; how long was it held?
$696 2C 4A   CI     #10   ; 10 passes or more?
$698 7F A4   BC     @skip4 ; that was a hold, not a tap
$69A 81      LDZ    #1    ; point at CDCMD_LO
$69B 44      LHI     #4    ; row 4
$69C C4      LI     #4    ; tap: CD command $04
$69D 02      S           ; CDCMD_LO = 4
$69E EE      IND     ; step to CDCMD_HI
$69F C0      CLA     ; zero, for the store below
$6A0 02      S           ; CDCMD_HI = 0
$6A1 CE      LI     #14   ; AUS code 14 -> U75 scancode $74
$6A2 6D D5   JMP    f_IssueButton ; report a front-panel button press to both sides of the machine.
@skip4:
$6A4 8A      LDZ    #10   ; point at KEY2_2
$6A5 41      LHI     #1    ; row 1
$6A6 C1      LI     #1    ; mask bit 0
$6A7 E7      AND     ; AC = KEY2_2 AND %0001 -> keeps bit 0
$6A8 7E BB   BZ     @skip5 ; no hold pulse for key 2
$6AA C1      LI     #1    ; 1 = key 2
$6AB AF 8E   CAL    f_ArmRepeat ; start a burst of three

```

f_SkipBack – hold action for key 2: CD command \$03, AUS 11 (\$77 on U75). Sets HOLD2_FIRED so the release will not also send the tap. Entered by falling through from \$6AB, and called again by f_RepeatTick for the rest of the burst.

```

f_SkipBack:
$6AD 81      LDZ    #1    ; point at CDCMD_LO
$6AE 44      LHI     #4    ; row 4
$6AF C3      LI     #3    ; CD command 3
$6B0 02      S           ; CDCMD_LO = 3
$6B1 EE      IND     ; step to CDCMD_HI
$6B2 C0      CLA     ; zero, for the store below
$6B3 02      S           ; CDCMD_HI = 0
$6B4 C1      LI     #1    ; AC = 1
$6B5 85      LDZ    #5    ; point at HOLD2_FIRED
$6B6 44      LHI     #4    ; row 4
$6B7 02      S           ; the hold action fired - the release must not send the tap
$6B8 CB      LI     #11   ; AUS code 11 -> U75 scancode $77
$6B9 6D D5   JMP    f_IssueButton ; report a front-panel button press to both sides of the machine.
@skip5:
$6BB 8E      LDZ    #14   ; point at KEY3_2
$6BC 41      LHI     #1    ; row 1
$6BD C1      LI     #1    ; mask bit 0
$6BE E7      AND     ; AC = KEY3_2 AND %0001 -> keeps bit 0
$6BF 7E D3   BZ     @skip6 ; no hold pulse for key 3
$6C1 C2      LI     #2    ; 2 = key 3
$6C2 AF 8E   CAL    f_ArmRepeat ; start a burst of three

```

f_SkipForward – hold action for key 3: CD command \$05, AUS 7 (\$76 on U75). Sets HOLD3_FIRED. Entered by falling through from \$6C2, and called again by f_RepeatTick for the rest of the burst.

```

f_SkipForward:
$6C4 81      LDZ    #1    ; point at CDCMD_LO
$6C5 44      LHI     #4    ; row 4
$6C6 C5      LI     #5    ; CD command 5
$6C7 02      S           ; CDCMD_LO = 5
$6C8 EE      IND     ; step to CDCMD_HI
$6C9 C0      CLA     ; zero, for the store below
$6CA 02      S           ; CDCMD_HI = 0
$6CB C1      LI     #1    ; AC = 1
$6CC 87      LDZ    #7    ; point at HOLD3_FIRED
$6CD 44      LHI     #4    ; row 4
$6CE 02      S           ; the hold action fired - the release must not send the tap
$6CF C7      LI     #7    ; AUS code 7 -> U75 scancode $76
$6D0 6D D5   JMP    f_IssueButton ; report a front-panel button press to both sides of the machine.
@done:
$6D2 62      RT       ; return
@skip6:
$6D3 88      LDZ    #8    ; point at KEY2_0
$6D4 41      LHI     #1    ; row 1
$6D5 21      L           ; AC = KEY2_0
$6D6 3E DC   BNZ    @skip7 ; key 2 still down?
$6D8 85      LDZ    #5    ; point at HOLD2_FIRED
$6D9 44      LHI     #4    ; row 4
$6DA C0      CLA     ; zero, for the store below
$6DB 02      S           ; released: the next press may tap again

```

```
@skip7:
$6DC 8C LDZ #12 ; point at KEY3_0
$6DD 41 LHI #1 ; row 1
$6DE 21 L ; AC = KEY3_0
$6DF 3E E5 BNZ @skip8 ; key 3 still down?
$6E1 87 LDZ #7 ; point at HOLD3_FIRED
$6E2 44 LHI #4 ; row 4
$6E3 C0 CLA ; zero, for the store below
$6E4 02 S ; released: the next press may tap again

@skip8:
$6E5 89 LDZ #9 ; point at POLL_TICK
$6E6 45 LHI #5 ; row 5
$6E7 21 L ; AC = POLL_TICK
$6E8 0E INC ; count passes since the last track poll
$6E9 02 S ; POLL_TICK = AC
$6EA 2C 4A CI #10 ; every tenth pass
$6EC 3E D2 BNZ @done ; if AC != 10
$6EE C0 CLA ; restart the count
$6EF 02 S ; POLL_TICK = 0
$6F0 81 LDZ #1 ; point at CDCMD_LO
$6F1 44 LHI #4 ; row 4
$6F2 C0 CLA ; command $10: ask the drive for the track number
$6F3 02 S ; CDCMD_LO = 0
$6F4 EE IND ; step to CDCMD_HI
$6F5 C1 LI #1 ; AC = 1
$6F6 02 S ; CDCMD_HI = 1
$6F7 AC 1C CAL f_SendCDCommand ; send it
$6F9 AC 79 CAL f_Wait ; a nested busy-wait of about 4.2 ms
$6FB AC 51 CAL f_SendCDCommand2 ; second phase of the CD exchange, eight more SCK pulses
$6FD 6F CB JMP f_ReadTrackReply ; read the two nibbles the CD drive returned and turn them into the track
```

\$6FF–\$6FF · 1 byte · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

Binary to BCD \$700-\$763

Binary to BCD conversion, 100 entries, fetched by RTBL at \$772. Converts the 7-bit value the CD drive returns over SDATA/SCK into the two track-number digits TRK10 and TRK1. Entries 0-9 carry \$A in the high nibble - font index 10, the blank glyph - so the leading zero is suppressed and track 0 blanks the field entirely. This is NOT an ASCII table. A string search finds \$714-\$74F printable because BCD values 20-79 happen to overlap the printable range; the giveaway is the six-byte gap at every \$xA-\$xF, exactly where the invalid BCD digits fall. Only 0-99 exist. The index can express 0-127, and 100-127 would read the code at \$764 onward as font indices.

Addr	Bytes	Meaning
\$700	\$AA,\$A1,\$A2,\$A3,\$A4,\$A5,\$A6,\$A7,\$A8,\$A9	tracks 0-9: leading digit blanked, so track 0 shows nothing at all
\$70A	\$10,\$11,\$12,\$13,\$14,\$15,\$16,\$17,\$18,\$19	tracks 10-19 shown as 10 .. 19
\$714	\$20,\$21,\$22,\$23,\$24,\$25,\$26,\$27,\$28,\$29	tracks 20-29 shown as 20 .. 29
\$71E	\$30,\$31,\$32,\$33,\$34,\$35,\$36,\$37,\$38,\$39	tracks 30-39 shown as 30 .. 39
\$728	\$40,\$41,\$42,\$43,\$44,\$45,\$46,\$47,\$48,\$49	tracks 40-49 shown as 40 .. 49
\$732	\$50,\$51,\$52,\$53,\$54,\$55,\$56,\$57,\$58,\$59	tracks 50-59 shown as 50 .. 59
\$73C	\$60,\$61,\$62,\$63,\$64,\$65,\$66,\$67,\$68,\$69	tracks 60-69 shown as 60 .. 69
\$746	\$70,\$71,\$72,\$73,\$74,\$75,\$76,\$77,\$78,\$79	tracks 70-79 shown as 70 .. 79
\$750	\$80,\$81,\$82,\$83,\$84,\$85,\$86,\$87,\$88,\$89	tracks 80-89 shown as 80 .. 89
\$75A	\$90,\$91,\$92,\$93,\$94,\$95,\$96,\$97,\$98,\$99	tracks 90-99 shown as 90 .. 99

f_TrackToDigits \$764

TMP0:TMP1 hold the 7-bit value read back by \$7CB. E:AC index the BCD table at \$700, and the two font indices land in TRK10 and TRK1. With the main logic unpowered both digits are blanked instead.

Purpose turn the drive's reply into the two track digits on the panel.

Reached from \$7D8

```
$764 89 LDZ #9 ; select port J
$765 0C IP ; is the main logic powered?
$766 82 LDZ #2 ; point at MPS_TMP2
$767 46 LHI #6 ; row 6
$768 02 S ; MPS_TMP2 = AC
$769 C2 LI #2 ; mask bit 1
$76A E7 AND ; AC = MPS_TMP2 AND %0010 -> keeps bit 1
$76B 7E 79 BZ @skip ; if the masked bit is clear
$76D 89 LDZ #9 ; point at TMP0
$76E 21 L ; AC = TMP0
$76F 0D XAE ; swap AC and E
$770 EE IND ; step to TMP1
$771 21 L ; AC = TMP1
$772 63 RTBL ; look up the two track digits
$773 84 LDZ #4 ; point at TRK10
$774 02 S ; TRK10 = AC
$775 EE IND ; step to TRK1
$776 0D XAE ; swap AC and E
$777 02 S ; TRK1 = AC
$778 62 RT ; return
@skip:
$779 84 LDZ #4 ; point at TRK10
$77A CA LI #10 ; unpowered: blank both track digits
$77B 02 S ; TRK10 = 10
$77C EE IND ; step to TRK1
$77D 02 S ; TRK1 = 10
$77E 62 RT ; return
```

f_PowerSymbol \$77F

set TMP1 to 12 (a bar glyph) when the main logic is down, 0 when it is up. Called once, from the reset code at \$027, and TMP1 is overwritten by f_ReadClock before anything reads it - so this routine has no effect. Probably a leftover from an earlier display layout.

Reached from \$027

```
$77F 8A LDZ #10 ; point at TMP1
$780 C0 CLA ; zero, for the store below
$781 02 S ; TMP1 = 0
$782 89 LDZ #9 ; select port J
$783 0C IP ; is the main logic powered?
$784 89 LDZ #9 ; point at TMP0
$785 02 S ; TMP0 = AC
$786 C2 LI #2 ; mask bit 1
$787 E7 AND ; AC = TMP0 AND %0010 -> keeps bit 1
$788 3E 8D BNZ @done ; if the masked bit is set
$78A 8A LDZ #10 ; point at TMP1
$78B CC LI #12 ; AC = 12
$78C 02 S ; not powered: bar glyph (result is never used)
@done:
$78D 62 RT ; return
```

f_ReadTrackReply \$7CB

digits. Both reads go through the BANK prefix - see the note at \$41C. reg14 is the low nibble, reg15 the high, masked to three bits for a 0-127 index into the BCD table.

Purpose read the two nibbles the CD drive returned and turn them into the track

Reached from \$6FD

```
$7CB 8E LDZ #14 ; select port 0
$7CC FD BANK ; BANK - see the note at $41C
$7CD 0C IP ; read reg 14 through the BANK prefix - see $41C
$7CE 8A LDZ #10 ; point at TMP1
$7CF 02 S ; TMP1 = AC
$7D0 8F LDZ #15 ; select port P
$7D1 FD BANK ; BANK - see the note at $41C
$7D2 0C IP ; read reg 15 through the BANK prefix - see $41C
$7D3 89 LDZ #9 ; point at TMP0
$7D4 02 S ; TMP0 = AC
$7D5 C7 LI #7 ; mask bits 0,1,2
$7D6 E7 AND ; AC = TMP0 AND %0111 -> keeps bits 0,1,2
$7D7 02 S ; TMP0 = AC
$7D8 AF 64 CAL f_TrackToDigits ; turn the drive's reply into the two track digits on the panel.
$7DA 62 RT ; return
```

f_DrawVolumeBar \$278

ATTEN_HI:ATTEN_LO is an attenuation index - 0 is loudest, \$22 is silence. It is squeezed to three bits and used to fetch a mask from the table at \$270 (\$FF \$FE \$FC \$F8 \$F0 \$E0 \$C0 \$80), so the bar is full at maximum volume and shrinks as the attenuation rises. Each bit of that mask becomes the Sh segment of one digit, written into BAR0-7 for the ISR to pick up. The digits are not filled in order - the call sequence below is the physical order of the bar segments on the panel. With the main logic powered down the bar is cleared instead.

Purpose show the current volume as an eight-step bar across the panel.

Reached from \$03F

```

$278 89    LDZ    #9          ; select port J
$279 0C    IP          ; is the main logic powered?
$27A 89    LDZ    #9          ; point at TMP0
$27B 02    S          ; TMP0 = AC
$27C C2    LI     #2          ; mask bit 1
$27D E7    AND          ; AC = TMP0 AND %0010 -> keeps bit 1
$27E 7E C5 BZ     $2C5        ; no - blank the whole bar
$280 C7    LI     #7          ; AC = 7
$281 0D    XAE          ; swap AC and E
$282 89    LDZ    #9          ; point at ATTEN_HI
$283 43    LHI     #3          ; row 3
$284 21    L          ; AC = ATTEN_HI
$285 2C 42 CI     #2          ; attenuation $20 or more - near silence?
$287 3E 8C BNZ    @skip       ; if AC != 2
$289 C7    LI     #7          ; yes - a single bar segment
$28A 6A A0 JMP     @skip2      ; skip the other branch

@skip:
$28C 01    RAL          ; shift AC left, top bit into carry
$28D 01    RAL          ; shift AC left, top bit into carry
$28E 89    LDZ    #9          ; point at TMP0
$28F 02    S          ; TMP0 = AC
$290 C4    LI     #4          ; mask bit 2
$291 E7    AND          ; AC = TMP0 AND %0100 -> keeps bit 2
$292 02    S          ; TMP0 = AC
$293 88    LDZ    #8          ; point at ATTEN_LO
$294 43    LHI     #3          ; row 3
$295 21    L          ; AC = ATTEN_LO
$296 01    RAL          ; shift AC left, top bit into carry
$297 01    RAL          ; shift AC left, top bit into carry
$298 01    RAL          ; shift AC left, top bit into carry
$299 8A    LDZ    #10         ; point at TMP1
$29A 02    S          ; TMP1 = AC
$29B C3    LI     #3          ; mask bits 0,1
$29C E7    AND          ; AC = TMP1 AND %0011 -> keeps bits 0,1
$29D 02    S          ; TMP1 = AC
$29E 89    LDZ    #9          ; point at TMP0
$29F E5    OR          ; combine into a 0-7 level: 0 = loudest = full bar

@skip2:
$2A0 63    RTBL         ; fetch the bar mask for that level
$2A1 86    LDZ    #6          ; point at BAR6
$2A2 43    LHI     #3          ; row 3
$2A3 AA BB CAL     f_ShiftBarBit ; bar segment for digit 6
$2A5 80    LDZ    #0          ; point at BAR0
$2A6 43    LHI     #3          ; row 3
$2A7 AA BB CAL     f_ShiftBarBit ; digits 0 and 1
$2A9 AA BB CAL     f_ShiftBarBit ; take the next bar bit out of AC into this digit's Sh
$2AB 87    LDZ    #7          ; point at BAR7
$2AC 43    LHI     #3          ; row 3
$2AD AA BB CAL     f_ShiftBarBit ; digit 7
$2AF 0D    XAE          ; second half of the mask
$2B0 82    LDZ    #2          ; point at BAR2
$2B1 43    LHI     #3          ; row 3
$2B2 AA BB CAL     f_ShiftBarBit ; digits 2 to 5
$2B4 AA BB CAL     f_ShiftBarBit ; take the next bar bit out of AC into this digit's Sh
$2B6 AA BB CAL     f_ShiftBarBit ; take the next bar bit out of AC into this digit's Sh
$2B8 AA BB CAL     f_ShiftBarBit ; take the next bar bit out of AC into this digit's Sh
$2BA 62    RT          ; return

```

f_ShiftBarBit \$2BB

take the next bar bit out of AC into this digit's Sh segment, then step to the next digit.

Reached from \$2A3, \$2A7, \$2A9, \$2AD, \$2B2, \$2B4, \$2B6, \$2B8

```
$2BB E1 CLC ; clear carry before the shift
$2BC 01 RAL ; shift the next bar bit out into carry
$2BD 3F C2 BNC @skip ; if the bit was 0
$2BF 0B SMB 3 ; segment on
$2C0 EE IND ; next cell
$2C1 62 RT ; return
@skip:
$2C2 2B RMB 3 ; segment off
$2C3 EE IND ; next cell
$2C4 62 RT ; return
$2C5 80 LDZ #0 ; point at BAR0
$2C6 43 LHI #3 ; row 3
$2C7 C8 LI #8 ; AC = 8
@loop:
$2C8 2B RMB 3 ; clear this digit's bar segment
$2C9 EE IND ; next cell
$2CA 0F DEC ; AC - 1
$2CB 3E C8 BNZ @loop ; all eight digits
$2CD 62 RT ; return
```

f_VolumeKeys \$300

ATTEN_HI:ATTEN_LO is an attenuation index, so volume UP means decrementing it and volume DOWN means incrementing it. Key row 1 bit 1 (and IR CPCP 9) is up; key row 2 bit 1 (and CPCP 8) is down. The two nibbles carry and borrow between them, and the index is clamped at \$22, which is silence. Nothing happens unless the main logic is powered, and each change is pushed straight to the headphone attenuator via \$546. Pressing both directions at once is ignored.

Purpose step the volume up or down while a volume key is held.

Reached from \$052

```
$300 89 LDZ #9 ; select port J
$301 0C IP ; AC = port J (AUS/MPS)
$302 89 LDZ #9 ; point at TMP0
$303 02 S ; TMP0 = AC
$304 C2 LI #2 ; mask bit 1
$305 E7 AND ; AC = TMP0 AND %0010 -> keeps bit 1
$306 7E 5E BZ @done ; no power, no volume changes
$308 86 LDZ #6 ; point at KEY1_2
$309 41 LHI #1 ; row 1
$30A 21 L ; AC = KEY1_2
$30B 89 LDZ #9 ; point at TMP0
$30C 02 S ; TMP0 = AC
$30D C2 LI #2 ; mask bit 1
$30E E7 AND ; AC = TMP0 AND %0010 -> keeps bit 1
$30F 8A LDZ #10 ; point at KEY2_2
$310 41 LHI #1 ; row 1
$311 E7 AND ; AC = AC AND KEY2_2
$312 3E 5E BNZ @done ; both directions at once - ignore
$314 85 LDZ #5 ; point at KEY1_1
$315 41 LHI #1 ; row 1
$316 21 L ; AC = KEY1_1
$317 EE IND ; step to KEY1_2
$318 E5 OR ; volume UP key (row 1 bit 1) pressed or repeating
$319 89 LDZ #9 ; point at TMP0
$31A 40 LHI #0 ; row 0
$31B 02 S ; TMP0 = AC
$31C C2 LI #2 ; mask bit 1
$31D E7 AND ; AC = TMP0 AND %0010 -> keeps bit 1
$31E 7E 32 BZ @skip3 ; not pressed
$320 88 LDZ #8 ; point at ATTEN_LO
$321 43 LHI #3 ; row 3
$322 21 L ; AC = ATTEN_LO
$323 3E 2B BNZ @skip ; no borrow needed
$325 EE IND ; step to ATTEN_HI
$326 21 L ; AC = ATTEN_HI
$327 7E 30 BZ @skip2 ; if it is zero
$329 0F DEC ; borrow from the high nibble
$32A 02 S ; ATTEN_HI = AC
@skip:
$32B 88 LDZ #8 ; point at ATTEN_LO
$32C 43 LHI #3 ; row 3
$32D 21 L ; AC = ATTEN_LO
$32E 0F DEC ; less attenuation = louder
$32F 02 S ; ATTEN_LO = AC
@skip2:
$330 AD 46 CAL f_SetAttenuator ; send the new level to the attenuator
@skip3:
$332 89 LDZ #9 ; point at KEY2_1
$333 41 LHI #1 ; row 1
$334 21 L ; AC = KEY2_1
$335 EE IND ; step to KEY2_2
$336 E5 OR ; volume DOWN key (row 2 bit 1) pressed or repeating
$337 89 LDZ #9 ; point at TMP0
$338 40 LHI #0 ; row 0
$339 02 S ; TMP0 = AC
$33A C2 LI #2 ; mask bit 1
$33B E7 AND ; AC = TMP0 AND %0010 -> keeps bit 1
```

```

$33C 7E 5E BZ @done ; if the masked bit is clear
$33E 88 LDZ #8 ; point at ATTEN_LO
$33F 43 LHI #3 ; row 3
$340 21 L ; AC = ATTEN_LO
$341 0E INC ; more attenuation = quieter
$342 02 S ; ATTEN_LO = AC
$343 3E 49 BNZ @skip4 ; no carry
$345 EE IND ; step to ATTEN_HI
$346 21 L ; AC = ATTEN_HI
$347 0E INC ; carry into the high nibble
$348 02 S ; ATTEN_HI = AC

@skip4:
$349 88 LDZ #8 ; point at ATTEN_LO
$34A 43 LHI #3 ; row 3
$34B 21 L ; AC = ATTEN_LO
$34C 2C 43 CI #3 ; would that pass $22?
$34E 3E 5C BNZ @skip5 ; if AC != 3
$350 EE IND ; step to ATTEN_HI
$351 21 L ; AC = ATTEN_HI
$352 2C 42 CI #2 ; check the high nibble too
$354 3E 5C BNZ @skip5 ; if AC != 2
$356 88 LDZ #8 ; point at ATTEN_LO
$357 43 LHI #3 ; row 3
$358 C2 LI #2 ; AC = 2
$359 02 S ; clamp at $22 = silence
$35A EE IND ; step to ATTEN_HI
$35B 02 S ; ATTEN_HI = 2

@skip5:
$35C AD 46 CAL f_SetAttenuator ; send the new level to the attenuator
@done:
$35E 62 RT ; return

```

f_SetAttenuator \$546

The index ATTEN_HI:ATTEN_LO is doubled because the table at \$500 holds two bytes per level; two RTBL lookups collect the four-nibble attenuator word into ATT0-ATT3. Interrupts are disabled around the transfer so the display ISR cannot break the bit timing, 18 bits are shifted out through \$5B7, and _VST is pulsed to latch the new attenuation. This is the only routine in the ROM that turns interrupts off.

Purpose send the volume to the headphone attenuator on the port G three-wire bus.
Reached from \$233, \$330, \$35C, \$40F, \$419

```

$546 88 LDZ #8 ; point at ATTEN_LO
$547 43 LHI #3 ; row 3
$548 21 L ; attenuation index, low nibble
$549 89 LDZ #9 ; point at TMP0
$54A 02 S ; TMP0 = AC
$54B 89 LDZ #9 ; point at ATTEN_HI
$54C 43 LHI #3 ; row 3
$54D 21 L ; high nibble
$54E 8A LDZ #10 ; point at TMP1
$54F 02 S ; TMP1 = AC
$550 89 LDZ #9 ; point at TMP0
$551 21 L ; AC = TMP0
$552 E1 CLC ; clear carry before the shift
$553 01 RAL ; double it: the table holds two bytes per level
$554 02 S ; TMP0 = AC
$555 EE IND ; step to TMP1
$556 21 L ; AC = TMP1
$557 01 RAL ; shift AC left, top bit into carry
$558 02 S ; TMP1 = AC
$559 8A LDZ #10 ; point at TMP1
$55A 21 L ; AC = TMP1
$55B 0D XAE ; swap AC and E
$55C EF DED ; previous cell
$55D 21 L ; AC = TMP0
$55E 63 RTBL ; first half of the attenuator word from the level table
$55F 8C LDZ #12 ; point at ATT0
$560 43 LHI #3 ; row 3
$561 02 S ; ATT0 = AC
$562 EE IND ; step to ATT1
$563 0D XAE ; swap AC and E
$564 02 S ; ATT1 = AC
$565 8A LDZ #10 ; point at TMP1
$566 21 L ; AC = TMP1
$567 0D XAE ; swap AC and E
$568 EF DED ; previous cell
$569 21 L ; AC = TMP0
$56A 0E INC ; AC + 1
$56B 63 RTBL ; second half
$56C 8E LDZ #14 ; point at ATT2
$56D 43 LHI #3 ; row 3
$56E 02 S ; ATT2 = AC
$56F EE IND ; step to ATT3
$570 0D XAE ; swap AC and E
$571 02 S ; ATT3 = AC
$572 2C 91 RCTL $1 ; interrupts off - the display ISR must not break the bit timing
$574 8C LDZ #12 ; point at ATT0
$575 43 LHI #3 ; row 3
$576 21 L ; AC = ATT0
$577 AD B7 CAL f_ShiftAttBit ; bit 1 of 18 - the top bit of ATT0
$579 AD B7 CAL f_ShiftAttBit ; bit 2 of 18
$57B AD B7 CAL f_ShiftAttBit ; bit 3 of 18
$57D AD B7 CAL f_ShiftAttBit ; bit 4 of 18

```

```

$57F 8D    LDZ    #13          ; point at ATT1
$580 43    LHI    #3          ; row 3
$581 21    L          ; AC = ATT1
$582 AD B7  CAL    f_ShiftAttBit ; bit 5 of 18
$584 AD B7  CAL    f_ShiftAttBit ; bit 6 of 18
$586 AD B7  CAL    f_ShiftAttBit ; bit 7 of 18
$588 AD B7  CAL    f_ShiftAttBit ; bit 8 of 18
$58A 8E    LDZ    #14          ; point at ATT2
$58B 43    LHI    #3          ; row 3
$58C 21    L          ; AC = ATT2
$58D AD B7  CAL    f_ShiftAttBit ; bit 9 of 18
$58F AD B7  CAL    f_ShiftAttBit ; bit 10 of 18
$591 AD B7  CAL    f_ShiftAttBit ; bit 11 of 18
$593 AD B7  CAL    f_ShiftAttBit ; bit 12 of 18
$595 8F    LDZ    #15          ; point at ATT3
$596 43    LHI    #3          ; row 3
$597 21    L          ; AC = ATT3
$598 AD B7  CAL    f_ShiftAttBit ; bit 13 of 18
$59A AD B7  CAL    f_ShiftAttBit ; bit 14 of 18
$59C AD B7  CAL    f_ShiftAttBit ; bit 15 of 18
$59E AD B7  CAL    f_ShiftAttBit ; bit 16 of 18
$5A0 C0    CLA          ; two trailing zero bits
$5A1 AD B7  CAL    f_ShiftAttBit ; bit 17 of 18
$5A3 AD B7  CAL    f_ShiftAttBit ; bit 18 of 18
$5A5 86    LDZ    #6          ; point at ATTSHADOW
$5A6 42    LHI    #2          ; row 2
$5A7 0A    SMB    2          ; raise _VST to latch the new attenuation
$5A8 21    L          ; AC = ATTSHADOW
$5A9 86    LDZ    #6          ; select port G
$5AA 61    OP          ; port G (attenuator bus) = AC
$5AB 86    LDZ    #6          ; point at ATTSHADOW
$5AC 42    LHI    #2          ; row 2
$5AD 2A    RMB    2          ; drop _VST
$5AE 21    L          ; AC = ATTSHADOW
$5AF 86    LDZ    #6          ; select port G
$5B0 61    OP          ; port G (attenuator bus) = AC
$5B1 86    LDZ    #6          ; point at ATTSHADOW
$5B2 42    LHI    #2          ; row 2
$5B3 28    RMB    0          ; leave _VDATA low
$5B4 2C 81  SCTL   $1        ; interrupts back on
$5B6 62    RT          ; return

```

f_ShiftAttBit \$5B7

The bit is rotated out of AC into carry, written to _VDATA (PG0) through the ATTSHADOW byte, then _VCK (PG1) is raised and lowered. The attenuator samples on the falling edge. A2 preserves the caller's accumulator.

Purpose clock one bit onto the attenuator bus.

Reached from \$577, \$579, \$57B, \$57D, \$582, \$584, \$586, \$588, \$58D, \$58F, \$591, \$593, \$598, \$59A, \$59C, \$59E, \$5A1, \$5A3

```

$5B7 01    RAL          ; move the top bit of AC into carry
$5B8 86    LDZ    #6          ; point at ATTSHADOW
$5B9 42    LHI    #2          ; row 2
$5BA 3F BF  BNC    @skip      ; if no carry
$5BC 08    SMB    0          ; data bit 1 -> _VDATA high
$5BD 6D C0  JMP    @skip2     ; skip the other branch
@skip:
$5BF 28    RMB    0          ; data bit 0 -> _VDATA low
@skip2:
$5C0 E8    XA2          ; restore the caller's AC from A2
$5C1 86    LDZ    #6          ; point at ATTSHADOW
$5C2 42    LHI    #2          ; row 2
$5C3 21    L          ; AC = ATTSHADOW
$5C4 86    LDZ    #6          ; select port G
$5C5 61    OP          ; present the data bit
$5C6 86    LDZ    #6          ; point at ATTSHADOW
$5C7 42    LHI    #2          ; row 2
$5C8 09    SMB    1          ; raise _VCK
$5C9 21    L          ; AC = ATTSHADOW
$5CA 00    NOP          ; timing pad
$5CB 86    LDZ    #6          ; select port G
$5CC 61    OP          ; port G (attenuator bus) = AC
$5CD 86    LDZ    #6          ; point at ATTSHADOW
$5CE 42    LHI    #2          ; row 2
$5CF 29    RMB    1          ; drop _VCK - the attenuator samples on this edge
$5D0 21    L          ; AC = ATTSHADOW
$5D1 86    LDZ    #6          ; select port G
$5D2 61    OP          ; port G (attenuator bus) = AC
$5D3 E8    XA2          ; bring the rotated AC back for the next bit
$5D4 62    RT          ; return

```

Power, genlock and keyboard reset

f_KeyboardReset \$05F

The press edge of key row 0 bit 1 sets KBRST_HOLD to 2 and pulls _KBRESET (PG3) low. U75 command CPCP 10 would do the same through the virtual-key path, but the U75 firmware as shipped cannot produce 10 (its encoder tops out at 9), so from the remote this is latent. KBRST_HOLD then counts 2..15 once per service pass and wraps, which releases the line - a fixed pulse of about fourteen passes, ~470 ms, regardless of how long the key is held.

```
Purpose      pulse the Amiga's _KBRESET line from a front-panel key.
Reached from $056

$05F 80    LDZ    #0          ; point at KBRST_HOLD
$060 46    LHI    #6          ; row 6
$061 21    L      ; AC = KBRST_HOLD
$062 0E    INC      ; AC + 1
$063 02    S      ; KBRST_HOLD = AC
$064 3E 6B BNZ    @skip      ; if not zero
$066 86    LDZ    #6          ; hold-off expired: release _KBRESET (bit 3 high)
$067 42    LHI    #2          ; row 2
$068 C8    LI     #8          ; set bit 3
$069 E5    OR     ; AC = AC OR ATTSHADOW
$06A 02    S      ; ATTSHADOW = AC
@skip:
$06B 81    LDZ    #1          ; point at KEY0_1
$06C 41    LHI    #1          ; row 1
$06D C2    LI     #2          ; mask bit 1
$06E E7    AND      ; did the reset key (row 0 bit 1) go down this pass?
$06F 7E 7A BZ     @skip2     ; if the masked bit is clear
$071 80    LDZ    #0          ; point at KBRST_HOLD
$072 46    LHI    #6          ; row 6
$073 C2    LI     #2          ; start the ~470 ms pulse
$074 02    S      ; KBRST_HOLD = 2
$075 86    LDZ    #6          ; point at ATTSHADOW
$076 42    LHI    #2          ; row 2
$077 C7    LI     #7          ; mask bits 0,1,2
$078 E7    AND      ; assert _KBRESET by clearing bit 3
$079 02    S      ; ATTSHADOW = AC
@skip2:
$07A 86    LDZ    #6          ; point at ATTSHADOW
$07B 42    LHI    #2          ; row 2
$07C 21    L      ; AC = ATTSHADOW
$07D 86    LDZ    #6          ; select port G
$07E 61    OP      ; drive the attenuator/keyboard-reset lines
$07F 62    RT      ; return
```

Segment font \$080-\$08F

Segment font, 16 entries, fetched by RTBL at \$0AD with E = 8. Bit 0 = Sa through bit 7 = Sh, matching PL0-3 and PM0-3 in the pinout. RTBL puts the high nibble (Se-Sh) in AC and the low nibble (Sa-Sd) in E. Entries 10-13 are the blank and bar glyphs used for the AM/PM indicator, the blinking colon and leading-zero suppression. Entries 14 and 15 are hex 'E' and 'F'; nothing in this firmware can produce them, since every digit source is BCD.

Addr	Byte	hgfedcba	Segments	Glyph	Shows
\$080	\$3F	00111111	a b c d e f		'0'
\$081	\$06	00000110	b c		'1'
\$082	\$5B	01011011	a b d e g		'2'
\$083	\$4F	01001111	a b c d g		'3'
\$084	\$66	01100110	b c f g		'4'
\$085	\$6D	01101101	a c d f g		'5'
\$086	\$7D	0111101	a c d e f g		'6'
\$087	\$07	00000111	a b c		'7'
\$088	\$7F	01111111	a b c d e f g		'8'

\$089	\$6F	01101111	a b c d f g		'9'
\$08A	\$00	00000000	—		blank
\$08B	\$20	00100000	f		bar, Sf only
\$08C	\$10	00010000	e		bar, Se only
\$08D	\$30	00110000	e f		double bar, Se+Sf
\$08E	\$79	01111001	a d e f g		'E'
\$08F	\$71	01110001	a e f g		'F'

Grid table \$090-\$097

Grid table, 8 entries, fetched by RTBL at \$0C8 with E = 9. One bit set per entry. This is the only place the physical panel layout is recorded. By grid number: G1 COLON, G2 HRS1, G3 HRS10, G4 AMPM, G5 TRK1, G6 TRK10, G7 MIN1, G8 MIN10 - hours, minutes and track each sit on a pair of adjacent grids. Which end of the panel is G1 cannot be derived from the ROM.

Addr	Byte	G1 → G8	Meaning
\$090	\$04		HRS10 lights grid G3
\$091	\$02		HRS1 lights grid G2
\$092	\$80		MIN10 lights grid G8
\$093	\$40		MIN1 lights grid G7
\$094	\$20		TRK10 lights grid G6
\$095	\$10		TRK1 lights grid G5
\$096	\$08		AMPM lights grid G4
\$097	\$01		COLON lights grid G1

f_PowerControl \$208

MPS (Power Sense, PJ1) tells U62 whether the Amiga logic is actually up. On a change to powered, the genlock bits are cleared and the attenuation is set to \$12, a moderate listening level. On a change to unpowered the attenuation is set to \$22 - full attenuation, silence - so nothing comes out of the headphones with the machine off. POWER (PK0) is the output that switches the main logic on and off; GMS0/GMS1 share the same port, so the shadow byte PORTK is read-modify-written. With the logic down the genlock lines are forced to a fixed pattern.

Purpose run the CDTV's soft power switch.
Reached from \$04E

```
$208 89 LDZ #9 ; select port J
$209 0C IP ; sample MPS - is the main logic powered?
$20A 81 LDZ #1 ; point at MPS_TMP
$20B 46 LHI #6 ; row 6
$20C 02 S ; MPS_TMP = AC
$20D C2 LI #2 ; mask bit 1
$20E E7 AND ; AC = MPS_TMP AND %0010 -> keeps bit 1
$20F 02 S ; MPS_TMP = AC
$210 8A LDZ #10 ; point at MPS_PREV
$211 43 LHI #3 ; row 3
$212 F5 EXL ; compare against the last sample
$213 7E 3B BZ @skip3 ; unchanged, nothing to do
$215 81 LDZ #1 ; point at MPS_TMP
$216 46 LHI #6 ; row 6
$217 21 L ; AC = MPS_TMP
$218 7E 2C BZ @skip ; MPS = 0: the main logic has just gone away
$21A 8A LDZ #10 ; point at PORTK
$21B 42 LHI #2 ; row 2
$21C C1 LI #1 ; main logic has come up: keep POWER, clear the genlock bits
$21D E7 AND ; AC = PORTK AND %0001 -> keeps bit 0
$21E 02 S ; PORTK = AC
$21F 88 LDZ #8 ; point at ATTEN_LO
$220 43 LHI #3 ; row 3
$221 C2 LI #2 ; attenuation $12 - a moderate listening level
$222 02 S ; ATTEN_LO = 2
$223 EE IND ; step to ATTEN_HI
$224 C1 LI #1 ; AC = 1
```

```
$225 02      S                ; ATTEN_HI = 1
$226 87      LDZ      #7        ; point at GMSMODE
$227 45      LHI      #5        ; row 5
$228 C0      CLA                ; zero, for the store below
$229 02      S                ; GMSMODE = 0
$22A 6A 33   JMP      @skip2    ; skip the other branch

@skip:
$22C 88      LDZ      #8        ; point at ATTEN_LO
$22D 43      LHI      #3        ; row 3
$22E C2      LI       #2        ; AC = 2
$22F 02      S                ; main logic has gone away: attenuation $22 = silence
$230 EE      IND                ; step to ATTEN_HI
$231 C2      LI       #2        ; AC = 2
$232 02      S                ; ATTEN_HI = 2

@skip2:
$233 AD 46   CAL      f_SetAttenuator ; push the volume to the headphone attenuator
$235 81      LDZ      #1        ; point at MPS_TMP
$236 46      LHI      #6        ; row 6
$237 21      L                ; AC = MPS_TMP
$238 8A      LDZ      #10       ; point at MPS_PREV
$239 43      LHI      #3        ; row 3
$23A 02      S                ; remember this MPS state for next time

@skip3:
$23B 86      LDZ      #6        ; point at SYSREQ_NEW
$23C 45      LHI      #5        ; row 5
$23D C2      LI       #2        ; mask bit 1
$23E E7      AND                ; did U75 ask for a power change?
$23F 7E 46   BZ       @skip4    ; if the masked bit is clear
$241 8A      LDZ      #10       ; point at PORTK
$242 42      LHI      #2        ; row 2
$243 C1      LI       #1        ; toggle bit 0
$244 F5      EXL                ; toggle the POWER bit
$245 02      S                ; PORTK = AC

@skip4:
$246 89      LDZ      #9        ; select port J
$247 0C      IP                ; AC = port J (AUS/MPS)
$248 81      LDZ      #1        ; point at MPS_TMP
$249 46      LHI      #6        ; row 6
$24A 02      S                ; MPS_TMP = AC
$24B C2      LI       #2        ; mask bit 1
$24C E7      AND                ; AC = MPS_TMP AND %0010 -> keeps bit 1
$24D 7E 60   BZ       @skip6    ; main logic is down
$24F 8D      LDZ      #13       ; point at KEY3_1
$250 41      LHI      #1        ; row 1
$251 C2      LI       #2        ; mask bit 1
$252 E7      AND                ; AC = KEY3_1 AND %0010 -> keeps bit 1
$253 7E 5A   BZ       @skip5    ; if the masked bit is clear
$255 8A      LDZ      #10       ; point at PORTK
$256 42      LHI      #2        ; row 2
$257 C8      LI       #8        ; toggle bit 3
$258 F5      EXL                ; toggle U60 pin 19
$259 02      S                ; PORTK = AC

@skip5:
$25A 8A      LDZ      #10       ; point at PORTK
$25B 42      LHI      #2        ; row 2
$25C 21      L                ; AC = PORTK
$25D 8A      LDZ      #10       ; select port K
$25E 61      OP                ; drive POWER and the genlock mode lines
$25F 62      RT                ; return

@skip6:
$260 8A      LDZ      #10       ; point at MPS_PREV
$261 43      LHI      #3        ; row 3
$262 02      S                ; MPS_PREV = AC
$263 8A      LDZ      #10       ; point at PORTK
$264 42      LHI      #2        ; row 2
$265 C1      LI       #1        ; mask bit 0
$266 E7      AND                ; AC = PORTK AND %0001 -> keeps bit 0
$267 02      S                ; PORTK = AC
$268 CC      LI       #12       ; set bits 2,3
$269 E5      OR                ; force the genlock lines to the powered-down pattern
$26A 02      S                ; PORTK = AC
$26B 8A      LDZ      #10       ; select port K
$26C 61      OP                ; drive POWER and the genlock mode lines
$26D 62      RT                ; return
```

\$26E–\$26F · 2 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

Volume bar masks \$270-\$277

Volume bar masks, fetched by RTBL at \$2A0. Indexed by (VOL >> 2) & 7, so the bar shrinks as the attenuation index rises. Each bit becomes the Sh segment of one digit via \$2BB; the fill order in \$2A1-\$2B8 is the physical left-to-right order of the bar on the panel. Note the bar never fully empties - even level 7 keeps one segment lit.

Addr	Byte	Bar	Meaning
\$270	\$FF		level 0: 8 of the 8 bar segments lit
\$271	\$FE		level 1: 7 of the 8 bar segments lit
\$272	\$FC		level 2: 6 of the 8 bar segments lit
\$273	\$F8		level 3: 5 of the 8 bar segments lit
\$274	\$F0		level 4: 4 of the 8 bar segments lit
\$275	\$E0		level 5: 3 of the 8 bar segments lit
\$276	\$C0		level 6: 2 of the 8 bar segments lit

f_ReadGenLockMode \$2D8

Both high means nothing is being requested, so GENLOCK_MS is cleared; anything else sets it and \$374 will act on it.

Purpose sample MS0/MS1, the genlock mode select lines the Amiga drives.

Reached from \$054

```
$2D8 88 LDZ #8 ; select port I
$2D9 0C IP ; read the Amiga genlock mode select lines MS0/MS1
$2DA 89 LDZ #9 ; point at TMP0
$2DB 02 S ; TMP0 = AC
$2DC C3 LI #3 ; mask bits 0,1
$2DD E7 AND ; keep MS0 and MS1
$2DE 2C 43 CI #3 ; both high means no genlock request
$2E0 3E E7 BNZ @skip ; if AC != 3
$2E2 8F LDZ #15 ; point at GENLOCK_MS
$2E3 45 LHI #5 ; row 5
$2E4 C0 CLA ; zero, for the store below
$2E5 02 S ; clear the genlock flag
$2E6 62 RT ; return

@skip:
$2E7 8F LDZ #15 ; point at GENLOCK_MS
$2E8 45 LHI #5 ; row 5
$2E9 C1 LI #1 ; AC = 1
$2EA 02 S ; set the genlock flag
$2EB 62 RT ; return
```

\$2EC–\$2EF · 4 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

\$2F0–\$2FF · 16 bytes · Unprogrammed EPROM. \$FF decodes as XD, but nothing reaches it.

f_GenLockMode \$374

Either U75 asks for it (REQ_NEW bit 0) or the Amiga asserts MS0/MS1. GMSMODE counts 0,1,2 and wraps; it is shifted up into the GMS bit positions and merged into the port K shadow alongside the POWER bit. Does nothing while the main logic is unpowered.

Purpose cycle the genlock mode when asked, and drive GMS0/GMS1.

Reached from \$04C

```
$374 89 LDZ #9 ; select port J
$375 0C IP ; AC = port J (AUS/MPS)
$376 89 LDZ #9 ; point at TMP0
$377 02 S ; TMP0 = AC
$378 C2 LI #2 ; mask bit 1
$379 E7 AND ; no power, nothing to do
$37A 7E 9F BZ @done ; if the masked bit is clear
$37C 8F LDZ #15 ; point at GENLOCK_MS
$37D 45 LHI #5 ; row 5
$37E C1 LI #1 ; mask bit 0
$37F E7 AND ; has the Amiga taken over genlock control?
$380 3E A0 BNZ @skip2 ; if the masked bit is set
$382 86 LDZ #6 ; point at SYSREQ_NEW
$383 45 LHI #5 ; row 5
$384 C1 LI #1 ; mask bit 0
$385 E7 AND ; did U75 request a genlock change?
$386 7E 9F BZ @done ; if the masked bit is clear
$388 AF DB CAL f_ClearGenlockBits ; clear the genlock bits in the port K shadow
$38A 87 LDZ #7 ; point at GMSMODE
$38B 45 LHI #5 ; row 5
$38C 21 L ; AC = GMSMODE
$38D 0E INC ; step the genlock mode

@loop:
$38E 02 S ; GMSMODE = AC
$38F C3 LI #3 ; mask bits 0,1
$390 E7 AND ; AC = GMSMODE AND %0011
$391 2C 43 CI #3 ; wrap after mode 2
$393 3E 96 BNZ @skip ; if AC != 3
$395 C0 CLA ; zero, for the store below

@skip:
$396 02 S ; GMSMODE = AC
$397 E1 CLC ; clear carry before the shift
$398 01 RAL ; shift the mode into the GMS bit positions
$399 8A LDZ #10 ; point at PORTK
$39A 42 LHI #2 ; row 2
$39B E5 OR ; merge with the POWER bit
$39C 02 S ; PORTK = AC
$39D 8A LDZ #10 ; select port K
$39E 61 OP ; drive GMS0/GMS1 out to the remote-control genlock

@done:
$39F 62 RT ; return

@skip2:
$3A0 88 LDZ #8 ; select port I
$3A1 0C IP ; read MS0/MS1 from the Amiga
$3A2 89 LDZ #9 ; point at TMP0
$3A3 02 S ; TMP0 = AC
```

```

$3A4 C3    LI    #3          ; mask bits 0,1
$3A5 E7    AND           ; AC = TMP0 AND %0011 -> keeps bits 0,1
$3A6 2C 43  CI    #3          ; both high means the Amiga is not asking
$3A8 7E 9F  BZ    @done       ; if AC == 3
$3AA AF DB  CAL    f_ClearGenlockBits ; mask the GMS bits out of the port K shadow, keeping
$3AC 89    LDZ    #9          ; point at TMP0
$3AD 21    L          ; AC = TMP0
$3AE 87    LDZ    #7          ; point at GMSMODE
$3AF 45    LHI    #5          ; row 5
$3B0 6B 8E  JMP    @loop      ; skip the other branch

```

f_ClearGenlockBits \$7DB

mask the GMS bits out of the port K shadow, keeping POWER and U60 pin 19.

Reached from \$388, \$3AA

```

$7DB 8A    LDZ    #10         ; point at PORTK
$7DC 42    LHI    #2          ; row 2
$7DD C9    LI     #9          ; clear the genlock mode bits, keep POWER and U60 pin 19
$7DE E7    AND           ; AC = PORTK AND %1001 -> keeps bits 0,3
$7DF 02    S          ; PORTK = AC
$7E0 62    RT          ; return

```

\$7E1–\$7EF · 15 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

U75 interface

f_LatchU75Requests \$35F

REQ_NEW = REQ4 AND NOT REQ_PREV, so a request only fires on the pass where it first appears. \$208 and \$374 consume the result.

Purpose turn the level-held U75 request bits into one-shot edges.
Reached from \$04A

```
$35F 85 LDZ #5 ; point at SYSREQ_PREV
$360 45 LHI #5 ; row 5
$361 21 L ; AC = SYSREQ_PREV
$362 89 LDZ #9 ; point at TMP0
$363 02 S ; TMP0 = AC
$364 CF LI #15 ; toggle bits 0,1,2,3
$365 F5 EXL ; invert the previous request bits
$366 02 S ; TMP0 = AC
$367 84 LDZ #4 ; point at SYSREQ
$368 45 LHI #5 ; row 5
$369 21 L ; AC = SYSREQ
$36A EE IND ; step to SYSREQ_PREV
$36B 02 S ; this sample becomes the previous one
$36C 89 LDZ #9 ; point at TMP0
$36D 21 L ; AC = TMP0
$36E 85 LDZ #5 ; point at SYSREQ_PREV
$36F 45 LHI #5 ; row 5
$370 E7 AND ; newly-asserted bits only - a rising-edge detector
$371 EE IND ; step to SYSREQ_NEW
$372 02 S ; store the freshly-asserted U75 requests
$373 62 RT ; return
```

f_ReadU75Command \$48A

U75 puts a 4-bit code on CPCP0-3 (its PB4-7, our port A). The five request cells REQ0-REQ4 are cleared first, then the code selects one of them and sets it to 1 or 2. Code 0 means "nothing pending", and any code above 10 falls through to the RT with every cell already cleared - so 11-15 act as an explicit release. VKEY0-VKEY3 are injected into key-matrix rows 0-3 by \$104: value 1 sets that row's bit 0, value 2 sets bit 1. So eight of the eleven codes are virtual button presses. SYSREQ (code 2 or 3) is the exception - \$35F edge-detects it into genlock and power actions.

CPCP	cell = value	effect
1	VKEY3 = 2	row 3 bit 1 - CD/TV, toggles U60 pin 19 in \$208
2	SYSREQ = 1	genlock mode step (\$374)
3	SYSREQ = 2	power toggle (\$208)
4	VKEY3 = 1	row 3 bit 0 - previous track / scan back
5	VKEY0 = 1	row 0 bit 0 - play/pause
6	VKEY1 = 1	row 1 bit 0 - stop
7	VKEY2 = 1	row 2 bit 0 - next track / scan forward
8	VKEY2 = 2	row 2 bit 1 - volume UP
9	VKEY1 = 2	row 1 bit 1 - volume DOWN
10	VKEY0 = 2	row 0 bit 1 - keyboard reset (\$05F). Accepted here, but never sent: U75's f_MediaKeyCPCP cannot produce 10

This is the whole of the U75 -> U62 direction. U75 asserts a code once per IR frame and clears it within 56 ms, so a virtual key is never held long enough (10 passes, 335 ms) to trigger a hold action - the remote can only tap.

Purpose read what the Amiga side is asking for.
Reached from \$046

```
$48A 84 LDZ #4 ; point at SYSREQ
$48B 45 LHI #5 ; row 5
$48C C0 CLA ; zero, for the store below
$48D 02 S ; SYSREQ = 0
$48E 80 LDZ #0 ; point at VKEY0
$48F 45 LHI #5 ; row 5
$490 02 S ; VKEY0 = 0
$491 EE IND ; step to VKEY1
$492 02 S ; VKEY1 = 0
$493 EE IND ; step to VKEY2
$494 02 S ; VKEY2 = 0
$495 EE IND ; step to VKEY3
$496 02 S ; VKEY3 = 0
$497 80 LDZ #0 ; select port A
$498 0C IP ; AC = port A (CPCP from U75)
$499 7E C3 BZ @done ; if zero
$49B 2C 41 CI #1 ; is AC 1?
$49D 7E C4 BZ @skip ; if AC == 1
$49F 2C 42 CI #2 ; is AC 2?
$4A1 7E C9 BZ @skip2 ; if AC == 2
$4A3 2C 43 CI #3 ; is AC 3?
$4A5 7E CE BZ @skip3 ; if AC == 3
$4A7 2C 44 CI #4 ; is AC 4?
$4A9 7E D3 BZ @skip4 ; if AC == 4
$4AB 2C 45 CI #5 ; is AC 5?
$4AD 7E D8 BZ @skip5 ; if AC == 5
$4AF 2C 46 CI #6 ; is AC 6?
$4B1 7E DD BZ @skip6 ; if AC == 6
$4B3 2C 47 CI #7 ; is AC 7?
$4B5 7E E2 BZ @skip7 ; if AC == 7
$4B7 2C 48 CI #8 ; is AC 8?
```

```
$4B9 7E EC BZ @skip9 ; if AC == 8
$4BB 2C 49 CI #9 ; is AC 9?
$4BD 7E E7 BZ @skip8 ; if AC == 9
$4BF 2C 4A CI #10 ; is AC 10?
$4C1 7E F1 BZ @skip10 ; if AC == 10

@done:
$4C3 62 RT ; return

@skip:
$4C4 83 LDZ #3 ; point at VKEY3
$4C5 45 LHI #5 ; row 5
$4C6 C2 LI #2 ; AC = 2
$4C7 02 S ; VKEY3 = 2
$4C8 62 RT ; return

@skip2:
$4C9 84 LDZ #4 ; point at SYSREQ
$4CA 45 LHI #5 ; row 5
$4CB C1 LI #1 ; AC = 1
$4CC 02 S ; SYSREQ = 1
$4CD 62 RT ; return

@skip3:
$4CE 84 LDZ #4 ; point at SYSREQ
$4CF 45 LHI #5 ; row 5
$4D0 C2 LI #2 ; AC = 2
$4D1 02 S ; SYSREQ = 2
$4D2 62 RT ; return

@skip4:
$4D3 83 LDZ #3 ; point at VKEY3
$4D4 45 LHI #5 ; row 5
$4D5 C1 LI #1 ; AC = 1
$4D6 02 S ; VKEY3 = 1
$4D7 62 RT ; return

@skip5:
$4D8 80 LDZ #0 ; point at VKEY0
$4D9 45 LHI #5 ; row 5
$4DA C1 LI #1 ; AC = 1
$4DB 02 S ; VKEY0 = 1
$4DC 62 RT ; return

@skip6:
$4DD 81 LDZ #1 ; point at VKEY1
$4DE 45 LHI #5 ; row 5
$4DF C1 LI #1 ; AC = 1
$4E0 02 S ; VKEY1 = 1
$4E1 62 RT ; return

@skip7:
$4E2 82 LDZ #2 ; point at VKEY2
$4E3 45 LHI #5 ; row 5
$4E4 C1 LI #1 ; AC = 1
$4E5 02 S ; VKEY2 = 1
$4E6 62 RT ; return

@skip8:
$4E7 81 LDZ #1 ; point at VKEY1
$4E8 45 LHI #5 ; row 5
$4E9 C2 LI #2 ; AC = 2
$4EA 02 S ; VKEY1 = 2
$4EB 62 RT ; return

@skip9:
$4EC 82 LDZ #2 ; point at VKEY2
$4ED 45 LHI #5 ; row 5
$4EE C2 LI #2 ; AC = 2
$4EF 02 S ; VKEY2 = 2
$4F0 62 RT ; return

@skip10:
$4F1 80 LDZ #0 ; point at VKEY0
$4F2 45 LHI #5 ; row 5
$4F3 C2 LI #2 ; AC = 2
$4F4 02 S ; VKEY0 = 2
$4F5 62 RT ; return
```

\$4F6–\$4FF · 10 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

Attenuator words \$500-\$545

Headphone attenuator words, 35 pairs, fetched by the two RTBL calls in \$546. Indexed by twice the volume, which is why the clamp in \$300 is \$22. Layout, shifted out MSB first by \$5B7 as ATT0 ATT1 ATT2 ATT3 then two zero bits: byte 1: c c c c c c c f byte 2: f f f 0 1 1 0 7 6 5 4 3 2 1 0 7 6 5 4 Bits 7-1 of byte 1 are a 7-way one-hot coarse tap, one position per five steps. Bit 0 of byte 1 plus bits 7-4 of byte 2 are a 5-way one-hot fine tap. 7 x 5 = 35. The low nibble of byte 2 is always \$6 - a constant address or mode field. ATTEN is an attenuation index: 0 is loudest, 34 is silence.

Addr	Bytes	Meaning
\$500	\$81,\$06	volume 0 coarse tap 7, fine tap 0
\$502	\$80,\$86	volume 1 coarse tap 7, fine tap 1
\$504	\$80,\$46	volume 2 coarse tap 7, fine tap 2
\$506	\$80,\$26	volume 3 coarse tap 7, fine tap 3
\$508	\$80,\$16	volume 4 coarse tap 7, fine tap 4
\$50A	\$41,\$06	volume 5 coarse tap 6, fine tap 0
\$50C	\$40,\$86	volume 6 coarse tap 6, fine tap 1
\$50E	\$40,\$46	volume 7 coarse tap 6, fine tap 2
\$510	\$40,\$26	volume 8 coarse tap 6, fine tap 3
\$512	\$40,\$16	volume 9 coarse tap 6, fine tap 4
\$514	\$21,\$06	volume 10 coarse tap 5, fine tap 0
\$516	\$20,\$86	volume 11 coarse tap 5, fine tap 1

\$518	\$20, \$46	volume 12 coarse tap 5, fine tap 2
\$51A	\$20, \$26	volume 13 coarse tap 5, fine tap 3
\$51C	\$20, \$16	volume 14 coarse tap 5, fine tap 4
\$51E	\$11, \$06	volume 15 coarse tap 4, fine tap 0
\$520	\$10, \$86	volume 16 coarse tap 4, fine tap 1
\$522	\$10, \$46	volume 17 coarse tap 4, fine tap 2
\$524	\$10, \$26	volume 18 coarse tap 4, fine tap 3
\$526	\$10, \$16	volume 19 coarse tap 4, fine tap 4
\$528	\$09, \$06	volume 20 coarse tap 3, fine tap 0
\$52A	\$08, \$86	volume 21 coarse tap 3, fine tap 1
\$52C	\$08, \$46	volume 22 coarse tap 3, fine tap 2
\$52E	\$08, \$26	volume 23 coarse tap 3, fine tap 3
\$530	\$08, \$16	volume 24 coarse tap 3, fine tap 4
\$532	\$05, \$06	volume 25 coarse tap 2, fine tap 0
\$534	\$04, \$86	volume 26 coarse tap 2, fine tap 1
\$536	\$04, \$46	volume 27 coarse tap 2, fine tap 2
\$538	\$04, \$26	volume 28 coarse tap 2, fine tap 3
\$53A	\$04, \$16	volume 29 coarse tap 2, fine tap 4
\$53C	\$03, \$06	volume 30 coarse tap 1, fine tap 0
\$53E	\$02, \$86	volume 31 coarse tap 1, fine tap 1
\$540	\$02, \$46	volume 32 coarse tap 1, fine tap 2
\$542	\$02, \$26	volume 33 coarse tap 1, fine tap 3
\$544	\$02, \$16	volume 34 coarse tap 1, fine tap 4

f_IssueButton \$5D5

AC holds the AUS event code, which goes out on port J to U75 and from there to the Amiga. CDCMD_LO/HI hold the matching transport command, which \$41C clocks out on SDATA/SCK. POLL_TICK is then set to 8 so the track number is re-read two passes later, in case the command changed it.

Purpose report a front-panel button press to both sides of the machine.
Reached from \$62D, \$640, \$671, \$6A2, \$6B9, \$6D0

```
$5D5 89 LDZ #9 ; point at PORTJ
$5D6 42 LHI #2 ; row 2
$5D7 02 S ; PORTJ = AC
$5D8 89 LDZ #9 ; select port J
$5D9 61 OP ; send the event code to U75 on AUS0-2
$5DA AC 1C CAL f_SendCDCommand ; send the matching command over SDATA/SCK
$5DC 89 LDZ #9 ; point at POLL_TICK
$5DD 45 LHI #5 ; row 5
$5DE C8 LI #8 ; AC = 8
$5DF 02 S ; re-read the track number two passes from now
$5E0 62 RT ; return
```

\$5E1–\$5EF · 15 bytes · Unused space between routines. \$00 is NOP, so execution falling in here runs harmlessly on to whatever follows.

\$5F0–\$5FF · 16 bytes · Unprogrammed EPROM. \$FF decodes as XD, but nothing reaches it.

The BANK prefix

\$41C, \$451 and \$7CB each precede a port access with a BANK instruction. The opcode is confirmed as \$FD against the datasheet scan, and BANK should only affect a following JMP — there is no JMP in any of them. But \$7CB reads through that prefix on a DPL value that otherwise selects port O, which is output only, and \$7D5 keeps three bits of the result while port P has a single pin. So BANK is altering the port decode in a way the datasheet does not describe. Every BANK in the ROM sits in one of those three routines, always immediately before a port access with DPL 12–15. Treat the port names on those lines as unreliable.

Front-panel button identities

The six transport buttons are identified by the CD command and AUS code they send, not by a label on the case. The tap/hold pairing is certain; which physical key is which is inferred.

Panel orientation

The grid table fixes which VFD grid each buffer digit lights, so the relative order is known. Which end of the panel is G1 cannot be derived from the ROM.